



UNIVERSITÀ DEGLI STUDI DI PADOVA
CORSO DI LAUREA IN INFORMATICA (L-31)

Corso di Ingegneria del Software
Anno Accademico 2025/2026

Specifica Tecnica

Gruppo: NightPRO

Sistema: SmartOrder

swe.nightpro@gmail.com

Data: 2026-04-16

Versione: 2.0

Tabella delle Versioni

Versione	Data	Autore/i	Descrizione delle Modifiche	Verificatore
2.0	2026-04-16	Davide Biasuzzi	Approvazione del documento	-
1.3	2026-04-15	Giovanni Ponso	Migliorata e allineato stile sezione frontend	Samuele Perozzo
1.2	2026-04-14	Samuele Perozzo	Migliorie alla documentazione dei pattern, all'architettura esagonale e frontend	Francesco Zanella
1.1	2026-04-13	Davide Biasuzzi	Revisione tecnologie, design patterns, architettura logica	Francesco Zanella
1.0	2026-04-09	Giovanni Ponso	Approvazione del documento	-
0.8	2026-04-09	Samuele Perozzo	Correzioni finali	Leonardo Bilato
0.7	2026-04-09	G. Ponso, S. Perozzo, F. Zanella	Raffinamento dell'architettura di dettaglio: standardizzazione della nomenclatura (API/Backend), ottimizzazione dello strato di Dominio e consolidamento degli stati di sessione lato Frontend. Aggiornamento contestuale di tutti i diagrammi UML.	Leonardo Bilato
0.6	2026-04-08	Davide Biasuzzi	Aggiornamento dello stato di implementazione dei requisiti e allineamento alla rimozione di un requisito	Leonardo Bilato
0.5	2026-04-04	Francesco Zanella	Raffinamento dell'architettura (Backend e Frontend) per ottimizzare flussi asincroni e separazione dei servizi; approfondimento analitico della sezione Database. Aggiornato tabella requisiti	Leonardo Bilato
0.4	2026-04-02	G. Ponso, L. Bilato	Stesura sezione Architettura di dettaglio	M. M. Romascu
0.3	2026-03-31	D. Biasuzzi, F. Zanella, S. Perozzo	Riorganizzazione documento, inserimento dei diagrammi PlantUML e stesura delle sezioni 4, 5, 6 e 7	M. M. Romascu
0.2	2026-03-26	Davide Biasuzzi	Inserimento paragrafo su esecuzione e su tecnologie utilizzate	Leonardo Bilato
0.1	2026-03-20	Davide Biasuzzi	Stesura base documento (Introduzione e Requisiti di Sistema)	Leonardo Bilato

Indice

Componenti del Gruppo	9
1 Introduzione	10
1.1 Scopo del documento	10
1.2 Scopo del prodotto	10
1.3 Glossario	10
1.4 Riferimenti	10
1.4.1 Riferimenti normativi	10
1.4.2 Riferimenti informativi	10
2 Tecnologie utilizzate	11
2.1 Linguaggi di programmazione	11
2.1.1 TypeScript	11
2.1.2 Python	11
2.1.3 SQL	11
2.2 Framework e librerie	11
2.2.1 Next.js	11
2.2.2 React	12
2.2.3 Tailwind CSS	12
2.2.4 FastAPI	12
2.2.5 Pydantic	12
2.2.6 OpenAI SDK	12
2.3 Strumenti di persistenza	12
2.3.1 PostgreSQL	12
2.3.2 psycopg2	13
2.4 Intelligenza artificiale	13
2.4.1 GPT-5.4	13
2.4.2 GPT-5.4-mini	13
2.4.3 Whisper	13
2.4.4 text-embedding-3-small	13
2.5 Infrastruttura e DevOps	13
2.5.1 Docker	13
2.5.2 Node.js	14
2.5.3 Uvicorn	14
2.5.4 ESLint	14
2.6 Dipendenze per categoria	15
2.6.1 Frontend – Core	15
2.6.2 Frontend – Styling	15
2.6.3 Frontend – Build e runtime	16
2.6.4 Backend – Framework e core	16
2.6.5 Backend – Linguaggio	16
2.6.6 Backend – Persistenza	17
2.6.7 Backend – AI	17
2.6.8 Infrastruttura e DevOps	18
3 Architettura di sistema	19
3.1 Obiettivi architetturali	19
3.2 Vista C4 - Livello 1 (System Context)	19
3.3 Vista C4 - Livello 2 (Container)	20

3.4	Stile architetturale adottato	20
4	Architettura di deployment	22
4.1	Architettura: Sottosistemi Monolitici Indipendenti	22
4.1.1	Motivazioni della scelta architettonica	22
4.2	Modularità e organizzazione dei sottosistemi	22
4.2.1	Isolamento verticale del Backend	23
4.2.2	Componenti di un modulo	23
4.3	Scenario di esecuzione	23
4.3.1	Ambiente di Produzione (Cloud)	23
4.3.2	Ambiente di Sviluppo e Manutenzione (Locale)	23
4.4	Nodi e responsabilità di deployment	24
4.5	Canali di comunicazione	24
4.6	Aspetti operativi e Continuous Deployment	25
4.7	Sicurezza e protezione dei dati	25
5	Architettura logica	27
5.1	Backend	27
5.1.1	Esigenze Architettureali	27
5.1.2	Soluzione Architettureale	27
5.1.3	Realizzazione	27
5.2	Frontend	28
5.2.1	Esigenze Architettureali	28
5.2.2	Soluzione Architettureale e Separazione delle Responsabilità	28
5.2.3	Realizzazione	29
5.2.4	Overview Globale dei Componenti	29
5.3	Design pattern adottati	29
5.3.1	Pattern backend	29
5.3.1.1	Dependency Injection	29
5.3.1.2	Principi SOLID rispettati	30
5.3.1.3	Repository	30
5.3.1.4	Principi SOLID rispettati	30
5.3.1.5	Adapter (Ports and Adapters)	31
5.3.1.6	Principi SOLID rispettati	31
5.3.2	Pattern frontend	32
5.3.2.1	Facade (Service Layer frontend)	32
5.3.2.2	Principi SOLID rispettati	32
5.3.2.3	Observer (SSE broadcaster + observer client)	33
5.3.2.4	Principi SOLID rispettati	33
5.3.2.5	Singleton (module-level services)	34
5.3.2.6	Principi SOLID rispettati	34
5.3.2.7	Strategie applicative: Optimistic UI e Debounce	34
6	Database	36
6.1	Ruolo del database nel sistema	36
6.2	Macro-aree informative	36
6.3	Struttura delle principali tabelle	37
6.3.0.1	Anagrafica Articoli (anaart)	37
6.3.0.2	Sessioni e Chat (chat_sessions, chat_messages)	37
6.3.0.3	Carrello (cart_items)	37
6.3.0.4	Ordini Finalizzati (orders, order_items)	37

6.3.0.5	Identità Temporanee e Persistenti (<code>app_users</code> , <code>anacli</code> , <code>asscli</code>)	38
6.3.0.6	Gestione Ticket e Lock Operatore (<code>tickets</code>)	38
6.3.0.7	Feedback e Ottimizzazione LLM (<code>message_feedbacks</code>)	38
6.3.0.8	Embedding Vettoriali per Ricerca Semantica (<code>product_embeddings</code>)	38
6.4	Uso dei log per analisi operativa e miglioramento	39
6.5	Strategie di indicizzazione e performance	39
6.6	Vincoli dati rilevanti	39
6.7	Output verso ERP	40
6.7.1	Struttura del payload JSON esportato	40
7	Architettura di dettaglio	42
7.1	Backend - C4 Livello 4: Classi e interfacce	42
7.1.1	Controllers (Inbound Ports)	42
7.1.1.1	AuthController	42
7.1.1.2	ConversationController	43
7.1.1.3	CartController	43
7.1.1.4	OrderController	44
7.1.1.5	TicketController	44
7.1.1.6	FeedbackController	45
7.1.1.7	ClientController	46
7.1.1.8	HealthController	46
7.1.1.9	ProductController	47
7.1.1.10	SSEController	47
7.1.2	Application Services (Use Cases)	48
7.1.2.1	AuthService	48
7.1.2.2	ConversationService	49
7.1.2.3	CartService	50
7.1.2.4	OrderService	50
7.1.2.5	TicketService	51
7.1.2.6	FeedbackService	52
7.1.2.7	ClientService	52
7.1.2.8	ProductService	53
7.1.2.9	SSEBroadcaster	53
7.1.2.10	ConfidenceScorer	54
7.1.3	Core Domain (Entities & Value Objects)	54
7.1.3.1	Cliente	54
7.1.3.2	Operatore	55
7.1.3.3	Order	56
7.1.3.4	Carrello	56
7.1.3.5	RigaCarrello	57
7.1.3.6	Session	58
7.1.3.7	Ticket	58
7.1.3.8	LogAI	59
7.1.4	Outbound Ports (Interfaces)	60
7.1.4.1	IUserRepository	60
7.1.4.2	ISessionManager	61
7.1.4.3	IOrderRepository	62
7.1.4.4	ITicketRepository	62
7.1.4.5	IAIClient	63
7.1.5	Infrastructure Adapters	64
7.1.5.1	PostgresAdapter	64

7.1.5.2	OpenAIAdapter	65
8	Architettura di Dettaglio - Frontend	65
8.1	Servizi di Accesso Dati (Service Facade)	65
8.1.0.1	cartService	65
8.1.0.2	chatService	66
8.1.0.3	authService	66
8.1.0.4	productService	67
8.1.0.5	orderService	67
8.1.0.6	ticketService	68
8.1.0.7	sessionService & Altri (<i>feedbackService, healthService, apiClient</i>)	69
8.2	Contesti e Stato Applicativo (React Contexts)	69
8.2.0.1	SessionContext	69
8.2.0.2	CartContext	70
8.3	Flussi Asincroni (Custom Hooks)	71
8.3.0.1	useSSE	71
8.3.0.2	useHealthCheck / useProductSearch	72
9	Stato dei requisiti funzionali	73
9.1	Riepilogo stato requisiti prestazionali e di vincolo	80

Elenco delle figure

1	Diagramma C4 L1: contesto del sistema SmartOrder.	19
2	Diagramma C4 L2: vista container.	20
3	Vista di deployment logico del sistema.	24
4	Architettura logica backend secondo il modello esagonale (Ports and Adapters). (Visualizza a risoluzione piena originale)	27
5	Component Diagram globale dell'architettura logica Frontend. (Visualizza a risoluzione piena originale)	29
6	Pattern Dependency Injection: composizione nel bootstrap e iniezione nei servizi applicativi.	30
7	Pattern Repository: AuthService dipende da IUserRepository, con implementazioni sostituibili.	31
8	Pattern Adapter applicato all'integrazione AI tramite OpenAIAdapter.	32
9	Pattern Facade nel frontend: cartService come interfaccia semplificata tra UI e client API.	33
10	Pattern Observer per propagazione eventi real-time via SSE.	33
11	Pattern Singleton nel frontend: cartService come servizio condiviso e CartProvider come sorgente unica dello stato carrello.	34
12	UML della classe AuthController.	42
13	UML della classe ConversationController.	43
14	UML della classe CartController.	44
15	UML della classe OrderController.	44
16	UML della classe TicketController.	45
17	UML della classe FeedbackController.	46
18	UML della classe ClientController.	46
19	UML della classe HealthController.	47
20	UML della classe ProductController.	47
21	UML della classe SSEController.	48

22	UML della classe AuthService.	49
23	UML della classe ConversationService.	49
24	UML della classe CartService.	50
25	UML della classe OrderService.	51
26	UML della classe TicketService.	52
27	UML della classe FeedbackService.	52
28	UML della classe ClientService.	53
29	UML della classe ProductService.	53
30	UML della classe SSEBroadcaster.	54
31	UML della classe ConfidenceScorer.	54
32	UML della classe Cliente.	55
33	UML della classe Operatore.	55
34	UML della classe Order.	56
35	UML della classe Carrello.	57
36	UML della classe RigaCarrello.	57
37	UML della classe Session.	58
38	UML della classe Ticket.	59
39	UML della classe LogAI.	60
40	UML dell'interfaccia IUserRepository.	61
41	UML dell'interfaccia ISessionManager.	62
42	UML dell'interfaccia IOrderRepository.	62
43	UML dell'interfaccia ITicketRepository.	63
44	UML dell'interfaccia IAIClient.	64
45	UML della classe PostgresAdapter.	64
46	UML della classe OpenAIAdapter.	65
47	UML del modulo cartService.	66
48	UML del modulo chatService.	66
49	UML del modulo authService.	67
50	UML del modulo productService.	67
51	UML del modulo orderService.	68
52	UML del modulo ticketService.	68
53	UML dei moduli minori e della libreria apiClient.	69
54	UML del Provider SessionContext.	70
55	UML del Provider CartContext.	71
56	UML del modulo hook useSSE.	71
57	UML dei restanti custom hook del frontend.	72
58	Riepilogo stato di implementazione dei requisiti funzionali.	73

Elenco delle tabelle

1	Componenti del gruppo NightPRO.	9
2	Dipendenze della categoria Frontend – Core.	15
3	Dipendenze della categoria Frontend – Styling.	15
4	Dipendenze della categoria Frontend – Build e runtime.	16
5	Dipendenze della categoria Backend – Framework e core.	16
6	Dipendenze della categoria Backend – Linguaggio.	16
7	Dipendenze della categoria Backend – Persistenza.	17
8	Dipendenze della categoria Backend – AI.	17
9	Dipendenze della categoria Infrastruttura e DevOps.	18
10	C4 L1 - Contesto del sistema SmartOrder.	19

11	C4 L2 - Container e responsabilita.	20
12	Nodi logici di deployment e relative responsabilità.	24
13	Aree logiche del modello dati relazionale.	36
14	Stato dei requisiti funzionali (fonte: Analisi dei Requisiti PB v2.2)	79
15	Riepilogo implementazione requisiti prestazionali e di vincolo	80

Componenti del Gruppo

Cognome	Nome	Matricola
Biasuzzi	Davide	2111000
Bilato	Leonardo	2071084
Zanella	Francesco	2116442
Romascu	Mihaela-Mariana	2079726
Ogniben	Michele	2042325
Perozzo	Samuele	2110989
Ponso	Giovanni	2000558

Tabella 1: Componenti del gruppo NightPRO.

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere l'architettura software del sistema *SmartOrder*, definendo le scelte tecnologiche, i pattern architetturali e progettuali adottati, e l'architettura di dettaglio di ciascun componente. La Specifica Tecnica costituisce il riferimento principale per la fase di codifica, garantendo coerenza tra progettazione e implementazione.

1.2 Scopo del prodotto

SmartOrder è una piattaforma B2B per l'automazione della gestione degli ordini commerciali. Il sistema consente ai clienti di formulare ordini in linguaggio naturale (testo, audio, immagini) e li converte automaticamente in ordini strutturati pronti per l'integrazione con sistemi ERP aziendali, riducendo errori e tempi di inserimento manuale.

1.3 Glossario

Per evitare ambiguità, i termini tecnici e di dominio utilizzati nel documento sono definiti nel *Glossario v2.0*. I termini presenti nel glossario sono contrassegnati con il simbolo _G alla prima occorrenza in ogni sezione.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- [Norme di Progetto v4.0](#) (Ultima modifica 2026-04-09)

1.4.2 Riferimenti informativi

- [Glossario v3.0](#) (Ultima modifica 2026-04-09)
- [Piano di Qualifica v3.0](#) (Ultima modifica 2026-04-09)
- [Next.js Documentation](#) (Ultima modifica 2026-03-20)
- [FastAPI Documentation](#) (Ultima modifica 2026-03-20)
- [OpenAI API Documentation](#) (Ultima modifica 2026-03-20)
- [PostgreSQL Documentation](#) (Ultima modifica 2026-03-20)
- [Software Architecture Patterns, SWE 2025-2026](#) (Ultima modifica 2026-03-26)
- [Dependency Injection, SWE 2025-2026](#) (Ultima modifica 2026-03-26)
- [Design Pattern Creazionali, SWE 2025-2026](#) (Ultima modifica 2026-03-26)
- [Design Pattern Strutturali, SWE 2025-2026](#) (Ultima modifica 2026-03-26)

2 Tecnologie utilizzate

In questa sezione vengono elencate le tecnologie adottate per lo sviluppo del sistema *SmartOrder*. Per ciascuna tecnologia viene indicata la versione utilizzata, la motivazione della scelta e il contesto d'impiego nel progetto.

2.1 Linguaggi di programmazione

2.1.1 TypeScript

- **Versione:** 5
- **Documentazione ufficiale:** [TypeScript Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Frontend
- **Contesto di utilizzo:** linguaggio principale per l'interfaccia utente dell'applicativo Next.js, sia per i componenti React che per la gestione dello stato lato client.
- **Vantaggi:** tipizzazione statica che riduce errori a runtime, ottimo supporto IDE, compatibilità nativa con React e Next.js.
- **Svantaggi:** overhead di compilazione rispetto a JavaScript puro.

2.1.2 Python

- **Versione:** 3.11
- **Documentazione ufficiale:** [Python 3 Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** implementazione della pipeline di elaborazione degli ordini tramite intelligenza artificiale e gestione delle API tramite FastAPI.
- **Vantaggi:** ecosistema maturo in ambito NLP/ML, compatibilità nativa con le librerie OpenAI, sintassi concisa, supporto asincrono con Uvicorn/ASGI.
- **Svantaggi:** prestazioni inferiori a linguaggi compilati; mitigato dall'uso di Uvicorn (ASGI asincrono).

2.1.3 SQL

- **Documentazione ufficiale:** [ISO/IEC 9075:2023 \(SQL\)](#) (Ultima modifica 2026-03-26)
- **Ambito:** Database
- **Contesto di utilizzo:** interrogazioni sul catalogo prodotti, gestione ordini, sessioni e carrelli.

2.2 Framework e librerie

2.2.1 Next.js

- **Versione:** 15.5
- **Documentazione ufficiale:** [Next.js Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Frontend full-stack
- **Contesto di utilizzo:** framework principale per il frontend. Gestisce il rendering dell'interfaccia utente (React) e la logica lato client.
- **Vantaggi:** App Router con Server Components, SSR e SSG, integrazione ottimale con React.
- **Svantaggi:** curva di apprendimento per le convenzioni App Router; vincolo a React come libreria UI.

2.2.2 React

- **Versione:** 19
- **Documentazione ufficiale:** [React Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** UI
- **Contesto di utilizzo:** costruzione dell'interfaccia utente tramite componenti dichiarativi, gestione dello stato con Context API e Hooks.
- **Vantaggi:** ecosistema vastissimo, modello a componenti riutilizzabili, Virtual DOM per aggiornamenti efficienti.

2.2.3 Tailwind CSS

- **Versione:** 4.1
- **Documentazione ufficiale:** [Tailwind CSS Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Styling
- **Contesto di utilizzo:** stilizzazione dei componenti React con approccio utility-first.
- **Vantaggi:** prototipazione rapida, design responsive_G integrato, nessun CSS custom da mantenere.
- **Svantaggi:** classi verbose nel markup; mitigato dalla composizione in componenti React.

2.2.4 FastAPI

- **Versione:** 0.104.0
- **Documentazione ufficiale:** [FastAPI Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** espone gli endpoint `/chat` e `/transcribe` per l'elaborazione AI, gestione degli ordini e delle sessioni. Funziona come backend principale dell'applicazione.
- **Vantaggi:** validazione automatica dei dati tramite Pydantic_G, documentazione OpenAPI auto-generata, supporto nativo `async/await`.
- **Svantaggi:** richiede un processo separato (Uvicorn) rispetto al runtime Node.js.

2.2.5 Pydantic

- **Versione:** 2.5.0
- **Documentazione ufficiale:** [Pydantic Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** definizione dei modelli di request/response per le API FastAPI, con validazione automatica dei tipi.

2.2.6 OpenAI SDK

- **Versione:** 1.6.0
- **Documentazione ufficiale:** [OpenAI API Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Backend / Modulo AI
- **Contesto di utilizzo:** SDK_G ufficiale per l'integrazione con GPT-5.4, GPT-5.4-mini, Whisper e text-embedding-3-small.

2.3 Strumenti di persistenza

2.3.1 PostgreSQL

- **Versione:** 16
- **Documentazione ufficiale:** [PostgreSQL 16 Documentation](#) (Ultima modifica 2026-03-26)

- **Ambito:** Database
- **Contesto di utilizzo:** unica base dati del sistema, accessibile dal modulo AI tramite Python (psycopg2). Tutte le operazioni sul catalogo prodotti, ordini, sessioni e carrelli avvengono tramite FastAPI.
- **Vantaggi:** robustezza, supporto a query complesse e full-text search, estensibilità (pgvector per Vector Search_G).

2.3.2 psycopg2

- **Versione:** $\geq 2.9.9$
- **Documentazione ufficiale:** [psycopg2 Documentation](#) (Ultima modifica 2026-03-26)
- **Ambito:** Modulo AI / Backend
- **Contesto di utilizzo:** driver Python per PostgreSQL con connection pooling_G (SimpleConnectionPool).

2.4 Intelligenza artificiale

2.4.1 GPT-5.4

- **Versione:** 2026-03-05
- **Documentazione ufficiale:** [OpenAI GPT-5.4](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello linguistico principale per la comprensione degli ordini, la classificazione degli intenti e le decisioni di business. Utilizzato con temperature=0.1 per risposte deterministiche. Natively multimodale: supporta analisi di immagini e documenti caricati dagli utenti per l'estrazione di testo tramite OCR e la comprensione del contenuto visivo (es. foto di ordini cartacei).

2.4.2 GPT-5.4-mini

- **Versione:** 2026-03-17
- **Documentazione ufficiale:** [OpenAI GPT-5.4-mini](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** modello leggero per l'estrazione rapida dei parametri di ricerca e la classificazione semantica degli intenti utente.

2.4.3 Whisper

- **Versione:** 1
- **Documentazione ufficiale:** [OpenAI Speech-to-Text \(Whisper\)](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** trascrizione di messaggi audio in testo, con prompt specifico per il dominio Ergon e lingua forzata a italiano.

2.4.4 text-embedding-3-small

- **Versione:** 3
- **Documentazione ufficiale:** [OpenAI Embeddings](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** generazione di vettori di embedding (1536 dimensioni) per l'indicizzazione semantica del catalogo prodotti e il confronto vettoriale con gli input utente tramite pgvector.

2.5 Infrastruttura e DevOps

2.5.1 Docker

- **Versione:** ≥ 24.0

- **Documentazione ufficiale:** [Docker Documentation](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** containerizzazione dell'intero sistema per garantire portabilità e riproducibilità degli ambienti. Docker Compose (≥ 2.20) orchestra l'avvio dei container.

2.5.2 Node.js

- **Versione:** 20
- **Documentazione ufficiale:** [Node.js 20 Documentation](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** runtime JavaScript per l'esecuzione di Next.js in sviluppo e produzione.

2.5.3 Uvicorn

- **Versione:** 0.24.0
- **Documentazione ufficiale:** [Uvicorn Documentation](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** server ASGI_G per l'esecuzione di FastAPI, con supporto a richieste asincrone.

2.5.4 ESLint

- **Versione:** 9
- **Documentazione ufficiale:** [ESLint Documentation](#) (Ultima modifica 2026-03-26)
- **Contesto di utilizzo:** linter_G statico per il codice TypeScript/JavaScript.

2.6 Dipendenze per categoria

Le seguenti tabelle organizzano le dipendenze per ambito tecnico, evidenziando per ciascun elemento il ruolo operativo nel progetto, il criterio di adozione rispetto ad alternative praticabili e l'integrazione con il resto dello stack applicativo.

2.6.1 Frontend – Core

Nome	Versione	Descrizione e motivazione d'uso
Next.js	≥ 15.5	Framework di riferimento per il frontend applicativo: e' stato adottato per fornire una struttura unificata di routing, rendering e organizzazione dei moduli invece di una configurazione manuale basata su tool separati; si integra in modo nativo con React 19 e TypeScript 5 nel ciclo di sviluppo e build.
React	≥ 19	Libreria di composizione dell'interfaccia utente: e' stata scelta per il modello a componenti e per la maturita' dell'ecosistema rispetto ad approcci meno diffusi nel contesto del progetto; costituisce il livello UI eseguito all'interno dell'applicazione Next.js.
TypeScript	≥ 5	Linguaggio principale del frontend: e' stato preferito a JavaScript non tipizzato per ridurre ambiguita' contrattuali tra moduli e facilitare la manutenzione evolutiva; si integra con React e Next.js tramite tipizzazione condivisa di componenti, payload e utilita'.

Tabella 2: Dipendenze della categoria Frontend – Core.

2.6.2 Frontend – Styling

Nome	Versione	Descrizione e motivazione d'uso
Tailwind CSS	≥ 4.1	Sistema di styling adottato per definire regole visuali consistenti direttamente nei componenti: e' stato scelto rispetto a una base CSS personalizzata estesa per ridurre divergenze tra pagine e accelerare la standardizzazione grafica; si integra nel toolchain Next.js/Node.js e nelle component library React del progetto.

Tabella 3: Dipendenze della categoria Frontend – Styling.

2.6.3 Frontend – Build e runtime

Nome	Versione	Descrizione e motivazione d'uso
Node.js	≥ 20	Runtime di esecuzione per la parte frontend e per i comandi di sviluppo/build: e' stato selezionato in quanto piattaforma di riferimento dell'ecosistema Next.js, evitando incompatibilita' di tooling presenti su versioni precedenti; costituisce il processo applicativo su cui vengono eseguiti i bundle React/TypeScript.
ESLint	≥ 9	Strumento di analisi statica del codice frontend: e' stato adottato per uniformare regole di qualita' e prevenire regressioni prima dell'integrazione, preferendolo a controlli esclusivamente manuali; si integra in locale con TypeScript per l'ecosistema di sviluppo.

Tabella 4: Dipendenze della categoria Frontend – Build e runtime.

2.6.4 Backend – Framework e core

Nome	Versione	Descrizione e motivazione d'uso
FastAPI	$\geq 0.104.0$	Framework di esposizione delle API applicative: e' stato scelto per la coerenza tra modellazione dei dati e gestione endpoint rispetto a framework Python piu' generici; si integra con Pydantic per la validazione e con Uvicorn come server di esecuzione ASGI.
Pydantic	$\geq 2.5.0$	Livello di modellazione e validazione dei dati nel backend: e' stato adottato per centralizzare i contratti applicativi in modo tipizzato invece di validazioni distribuite nei singoli handler; si integra direttamente con FastAPI nei modelli di richiesta/risposta e nei controlli di consistenza.
Uvicorn	$\geq 0.24.0$	Server ASGI per l'esecuzione del servizio backend: e' stato scelto come runtime leggero e allineato allo standard asincrono richiesto dal framework, in alternativa a server WSGI non adatti al medesimo modello concorrente; si integra con FastAPI come processo di avvio del servizio Python.

Tabella 5: Dipendenze della categoria Backend – Framework e core.

2.6.5 Backend – Linguaggio

Nome	Versione	Descrizione e motivazione d'uso
Python	≥ 3.11	Linguaggio di implementazione del backend e dei flussi AI: e' stato preferito per l'allineamento con le librerie richieste dal dominio applicativo e per la produttivita' nello sviluppo dei servizi; si integra con FastAPI/Uvicorn sul piano applicativo e con psycpg2/OpenAI SDK sul piano infrastrutturale.

Tabella 6: Dipendenze della categoria Backend – Linguaggio.

2.6.6 Backend – Persistenza

Nome	Versione	Descrizione e motivazione d'uso
PostgreSQL	≥ 16	DBMS relazionale unico del sistema: e' stato adottato per garantire consistenza transazionale e gestione strutturata dei dati operativi rispetto a soluzioni non relazionali non prioritarie per il dominio; si integra con il backend Python per persistenza di catalogo, ordini, sessioni e carrelli.
psycopg2	$\geq 2.9.9$	Driver di accesso a PostgreSQL nel backend: e' stato scelto per mantenere un'interfaccia diretta e stabile verso il DBMS, evitando astrazioni aggiuntive non necessarie al livello corrente di complessita'; si integra nei repository applicativi e nel meccanismo di connection pooling del servizio.

Tabella 7: Dipendenze della categoria Backend – Persistenza.

2.6.7 Backend – AI

Nome	Versione	Descrizione e motivazione d'uso
OpenAI SDK	$\geq 1.6.0$	Libreria di integrazione con i servizi AI: e' stata scelta come client ufficiale per ridurre il rischio di disallineamento rispetto alle API e semplificare la gestione delle chiamate; si integra nel backend FastAPI come adapter verso i modelli linguistici utilizzati dal dominio.
GPT-5.4	2026-03-05	Modello linguistico principale per la comprensione delle richieste e la generazione delle risposte operative: e' stato adottato per privilegiare accuratezza e coerenza nelle decisioni applicative rispetto a modelli piu' leggeri; si integra nel flusso orchestrato dal backend tramite OpenAI SDK.
GPT-5.4-mini	N/D	Modello linguistico ottimizzato per task a costo e latenza contenuti: e' stato scelto per le fasi di pre-elaborazione dove la velocita' e' prioritaria rispetto alla massima capacita' inferenziale; si integra come componente complementare a GPT-5.4 nelle pipeline di classificazione e instradamento.

Tabella 8: Dipendenze della categoria Backend – AI.

2.6.8 Infrastruttura e DevOps

Nome	Versione	Descrizione e motivazione d'uso
Docker	≥ 24.0	Piattaforma di containerizzazione degli artefatti applicativi: e' stata adottata per assicurare coerenza degli ambienti tra sviluppo, verifica e rilascio evitando configurazioni manuali eterogenee; si integra con i servizi frontend e backend come base comune di esecuzione.
Railway	N/D (servizio gestito)	Piattaforma cloud di Continuous Deployment (CD): automatizza il build e il deploy ad ogni aggiornamento sul repository di riferimento, garantendo aggiornamenti zero-downtime senza implementare pipeline CI locali per i test.

Tabella 9: Dipendenze della categoria Infrastruttura e DevOps.

3 Architettura di sistema

Questa sezione illustra l'architettura ad alto livello del sistema SmartOrder. Vengono definiti gli obiettivi progettuali che hanno guidato le scelte tecnologiche e viene fornita una rappresentazione progressiva del sistema utilizzando il modello C4, partendo dal contesto generale fino ad arrivare ai container principali. Infine, viene motivato lo stile architetturale adottato.

3.1 Obiettivi architetturali

L'architettura del sistema *SmartOrder* è stata definita per soddisfare i seguenti obiettivi:

- separare in modo netto responsabilità di presentazione, logica applicativa e persistenza;
- mantenere il backend indipendente dalla tecnologia client;
- supportare il flusso *Human-in-the-Loop* per i casi a bassa affidabilità AI;
- garantire integrazione con ERP tramite output JSON su cartella condivisa;
- favorire estendibilità verso nuovi canali di input (es. immagini, canali esterni).

3.2 Vista C4 - Livello 1 (System Context)

Il diagramma C4 Livello 1 individua i confini del sistema e gli attori esterni principali.

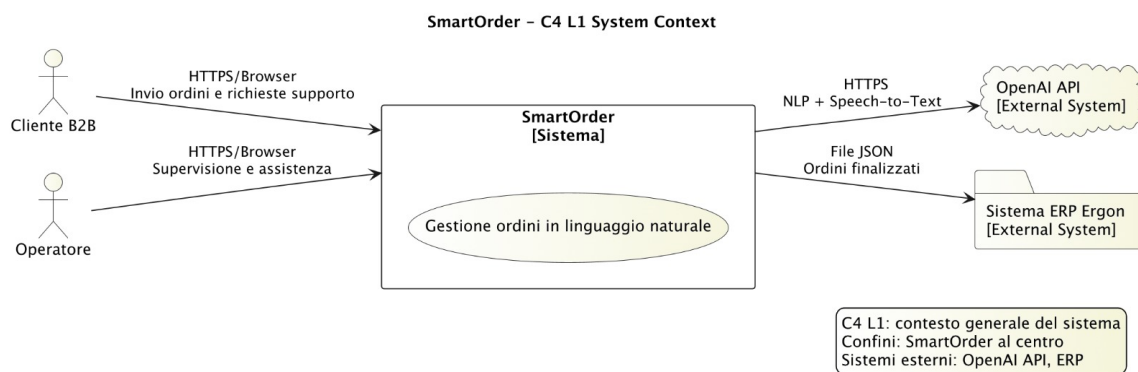


Figura 1: Diagramma C4 L1: contesto del sistema SmartOrder.

Elemento	Ruolo nel contesto
Cliente	Agente commerciale B2B che inserisce ordini in linguaggio naturale e richiede assistenza.
Operatore	Utente interno che supervisiona ticket, casi a bassa confidence e correzioni AI.
SmartOrder	Sistema oggetto della specifica; acquisisce input multimodale e genera ordini strutturati.
OpenAI API	Servizio esterno usato per comprensione linguaggio naturale e trascrizione audio.
Sistema ERP	Gestionale aziendale che riceve ordini finalizzati in formato JSON.

Tabella 10: C4 L1 - Contesto del sistema SmartOrder.

3.3 Vista C4 - Livello 2 (Container)

Il diagramma C4 Livello 2 decompone SmartOrder nei container applicativi principali.

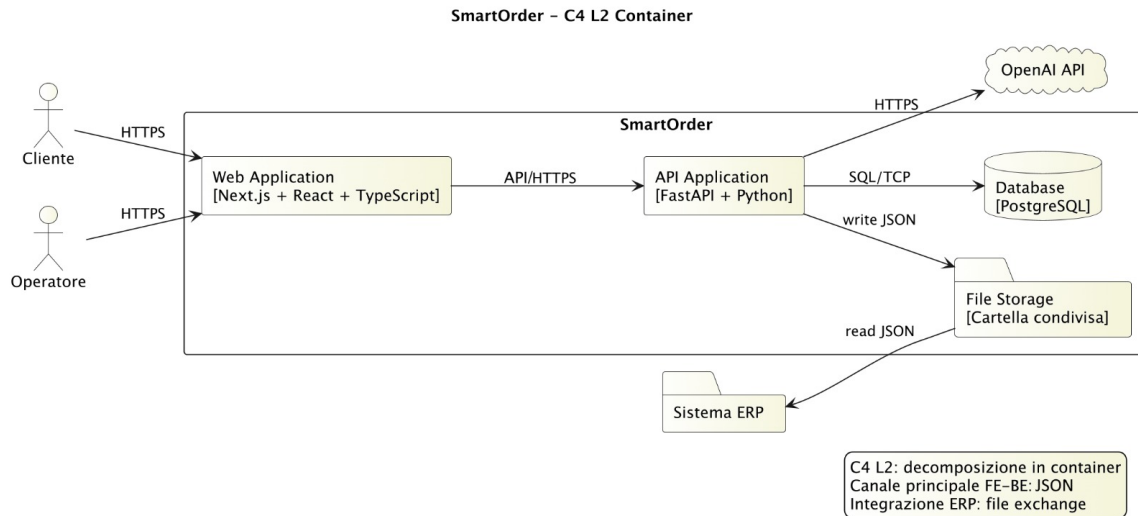


Figura 2: Diagramma C4 L2: vista container.

Container	Responsabilit�
Web Application (Next.js + React + TypeScript)	Interfaccia cliente/operatore, gestione sessione utente, rendering chat, carrello, storico, dashboard operative.
API Application (FastAPI)	Business logic di dominio: autenticazione, orchestrazione AI, gestione carrello/ordini/ticket, persistenza e pubblicazione output.
Database (PostgreSQL)	Persistenza strutturata di utenti, sessioni, carrelli, ordini, ticket, log AI e catalogo.
File Storage (cartella condivisa)	Punto di consegna ordini finalizzati in JSON verso processo ERP esterno.

Tabella 11: C4 L2 - Container e responsabilit .

3.4 Stile architetturale adottato

“La soluzione adotta un’architettura client-server composta da sottosistemi modulari, con frontend e backend separati e API come contratto principale. All’interno del backend viene applicata una suddivisione per layer logici (Controller, Application Service, Core Domain, Outbound Ports) coerente con i principi di *Ports and Adapters*.

Le principali motivazioni della scelta sono:

- isolamento della logica di business da dipendenze infrastrutturali;
- testabilit  delle regole applicative in assenza di interfaccia grafica;
- sostituibilit  di adapter esterni (servizi AI, storage, connettori DB) con impatto ridotto;

- evoluzione indipendente dei rilasci frontend e backend.

4 Architettura di deployment

Questa sezione descrive la distribuzione fisica dei componenti del sistema *SmartOrder*, illustrando gli ambienti di esecuzione, i nodi infrastrutturali coinvolti e le strategie di Continuous Deployment (CD_g).

4.1 Architettura: Sottosistemi Monolitici Indipendenti

Il sistema *SmartOrder* adotta un'architettura **Client-Server** basata su due unità di deployment indipendenti. Entrambi i componenti sono strutturati internamente come monoliti, garantendo una gestione semplificata del rilascio pur mantenendo una netta separazione delle responsabilità:

- **Backend (Modular Monolith):** Il server è un monolite modulare implementato in FastAPI (Python). Nonostante venga eseguito come un unico processo, la logica di business è isolata verticalmente per domini funzionali, assicurando coesione interna e facilità di manutenzione.
- **Frontend (Frontend Monolith):** Il client è un'applicazione Next.js distribuita come unità singola. Gestisce interamente lo strato di presentazione e lo stato dell'interfaccia utente, operando in un ambiente di runtime Node.js separato e dedicato.

Il disaccoppiamento tra i due sottosistemi è garantito dall'utilizzo di interfacce HTTP per lo scambio di dati in formato JSON. L'organizzazione del codice segue l'approccio **Monorepo**: entrambi i sottosistemi risiedono in un unico repository dei sorgenti. Questa scelta semplifica la gestione delle dipendenze e garantisce la coerenza tra le interfacce del frontend e le API del backend durante lo sviluppo.

4.1.1 Motivazioni della scelta architettonica

L'adozione di questa struttura garantisce un equilibrio ottimale tra semplicità operativa e flessibilità evolutiva, offrendo i seguenti vantaggi:

- **Efficienza Infrastrutturale:** L'approccio monolitico riduce drasticamente l'overhead di gestione rispetto a un'architettura a microservizi, pur mantenendo una chiara separazione delle responsabilità.
- **Scalabilità Indipendente:** La natura disaccoppiata dei due sottosistemi permette di scalare le risorse di calcolo in modo differenziato (es. aumentando le risorse del solo backend durante picchi di elaborazione AI).
- **Coerenza del Ciclo di Vita:** L'organizzazione in Monorepo assicura che il rilascio di nuove funzionalità avvenga in modo atomico, eliminando il rischio di disallineamento tra l'interfaccia utente e le logiche del server.

4.2 Modularità e organizzazione dei sottosistemi

Il sistema garantisce un elevato grado di manutenibilità grazie all'organizzazione modulare del codice, che permette di gestire la complessità dei due sottosistemi senza ricorrere a un'architettura a microservizi.

4.2.1 Isolamento verticale del Backend

Il backend è strutturato secondo i principi della **modularità verticale**. Ogni dominio di business (es. gestione ordini, carrello, ticket) è racchiuso in un modulo indipendente che contiene sia la logica applicativa che le interfacce per l'accesso ai dati. Questo isolamento permette di:

- Evitare dipendenze circolari tra le diverse funzionalità del sistema.
- Facilitare il collaudo dei singoli moduli in isolamento.
- Semplificare l'evoluzione di una specifica parte del sistema senza impatti sulle altre.

4.2.2 Componenti di un modulo

Per garantire uniformità e coesione, ogni modulo del backend si articola nei seguenti componenti logici:

1. **Controller**: Punto di ingresso per le richieste HTTP e validazione dell'input.
2. **Service**: Implementazione della logica di business e delle regole di dominio.
3. **Port**: Definizione astratta dei requisiti infrastrutturali (es. interfacce per il database).
4. **Schema**: Modelli di dati utilizzati per lo scambio di informazioni (DTO).
5. **Adapter**: Implementazione specifica delle Port per le tecnologie utilizzate (es. PostgreSQL, OpenAI).

4.3 Scenario di esecuzione

Il sistema è progettato per operare in due modalità distinte, garantendo sia l'accessibilità pubblica del servizio che la manutenibilità del codice sorgente.

4.3.1 Ambiente di Produzione (Cloud)

È l'ambiente principale in cui l'applicazione è effettivamente erogata agli utenti finali come **Web Application** accessibile via internet. L'intera infrastruttura è ospitata sulla piattaforma cloud **Railway**, che gestisce l'esecuzione dei seguenti servizi in container isolati:

- **Frontend Service**: Un'istanza Node.js che serve l'interfaccia web (Next.js) via protocollo sicuro HTTPS.
- **Backend Service**: Un'istanza Python che esegue il server FastAPI per l'elaborazione delle richieste API.
- **Database Service**: Un'istanza gestita di PostgreSQL per la persistenza dei dati transazionali.

4.3.2 Ambiente di Sviluppo e Manutenzione (Locale)

Questo scenario viene utilizzato esclusivamente per l'implementazione di nuove funzionalità o per attività di debugging. In questa modalità, l'applicazione viene eseguita localmente sulla macchina dello sviluppatore avviando i processi di frontend e backend in modo indipendente. La comunicazione avviene tramite l'interfaccia di rete locale (*localhost*), permettendo di testare le modifiche prima del rilascio definitivo in produzione.

4.4 Nodi e responsabilità di deployment

La seguente vista descrive la mappatura tra i componenti software e le risorse cloud che li ospitano in ambiente di produzione.

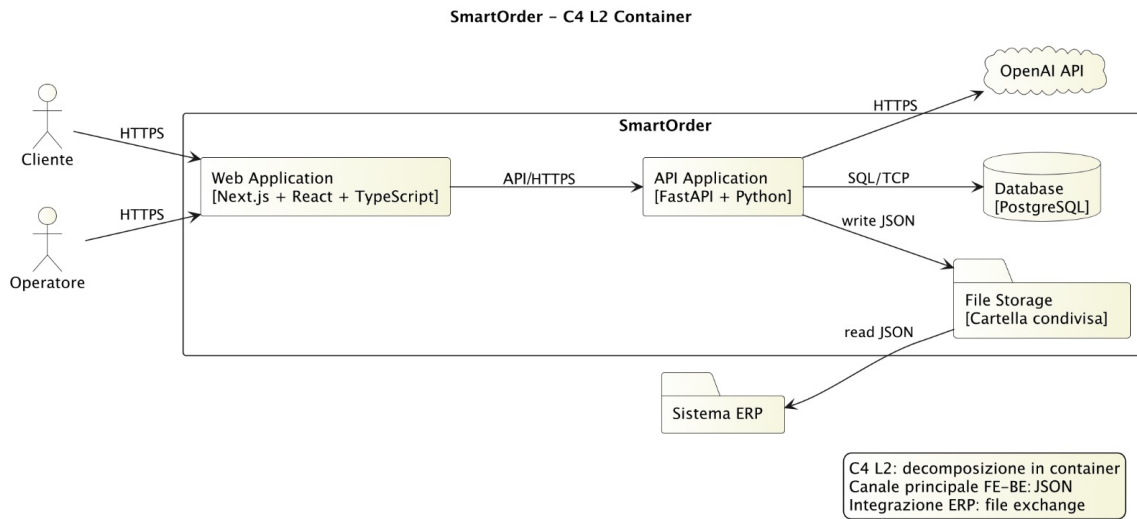


Figura 3: Vista di deployment logico del sistema.

Nodo Cloud	Responsabilità
Frontend Service (Railway)	Eroga l'applicazione Next.js tramite protocollo HTTPS. Gestisce il rendering dell'interfaccia utente e l'inoltro delle richieste API verso il backend.
Backend Service (Railway)	Esegue il server FastAPI (Uvicorn). Centralizza la logica di business, l'orchestrazione delle chiamate AI e la persistenza dei dati.
Database Service (Railway PostgreSQL)	Istanza gestita di PostgreSQL. Garantisce l'integrità e la persistenza dei dati transazionali relativi a utenti, ordini e sessioni.
File Storage Condiviso	Cartella accessibile nel File System utilizzata per depositare i file JSON degli ordini finalizzati, facilitando l'integrazione con il sistema ERP aziendale.
OpenAI Cloud API	Servizio esterno richiamato dal backend per l'elaborazione del linguaggio naturale (LLM), trascrizione vocale e riconoscimento immagini.

Tabella 12: Nodi logici di deployment e relative responsabilità.

4.5 Canali di comunicazione

Le interazioni tra i diversi componenti del sistema avvengono attraverso protocolli standardizzati, garantendo sicurezza e interoperabilità:

- **Frontend ↔ Backend:** La comunicazione avviene tramite chiamate API su protocollo cifrato HTTPS, con scambio di payload in formato JSON.
- **Backend ↔ PostgreSQL:** Il backend comunica con il database tramite protocollo TCP, ottimizzato attraverso un meccanismo di *connection pooling* per la gestione efficiente delle sessioni.
- **Backend ↔ OpenAI API:** Le richieste ai servizi di intelligenza artificiale vengono effettuate tramite invocazioni su canale sicuro HTTPS verso gli endpoint cloud di OpenAI.
- **Backend → File Storage:** L'integrazione con il sistema ERP avviene tramite operazioni di I/O su filesystem per il deposito dei file d'ordine in formato JSON.

4.6 Aspetti operativi e Continuous Deployment

Il ciclo di vita del software è gestito tramite pratiche automatizzate di **Continuous Deployment** (CD_g) per assicurare rilasci frequenti, coordinati e privi di disservizi.

- **Rilascio Unificato e Automatizzato:** Grazie all'adozione di un'architettura **Monorepo**, il processo di deployment è centralizzato. La piattaforma **Railway** è interfacciata direttamente con il repository GitHub e, ad ogni aggiornamento sul branch principale, avvia simultaneamente la build e il rilascio di entrambi i container (frontend e backend). Questo garantisce che le versioni dei due sottosistemi siano sempre sincronizzate e compatibili tra loro.
- **Aggiornamenti Zero-Downtime:** Il deployment avviene senza interruzioni di servizio per l'utente finale. I nuovi container vengono avviati e sostituiscono quelli precedenti solo dopo aver superato con successo i controlli di integrità del sistema, assicurando la massima disponibilità della piattaforma.
- **Gestione delle Configurazioni:** I dati sensibili, quali chiavi API e credenziali di accesso al database, non sono memorizzati nel codice sorgente. Vengono iniettati nei runtime come variabili d'ambiente criptate attraverso la dashboard di gestione della piattaforma cloud, rispettando i moderni standard di sicurezza.

4.7 Sicurezza e protezione dei dati

Nel progetto la sicurezza è stata considerata in modo continuo, integrandola nelle principali scelte architetturali. Di seguito sono riportate le misure adottate:

- **Difesa in profondità contro il furto di sessione:** autenticazione basata su cookie HTTPOnly e Secure, preferiti allo storage nel browser per ridurre la superficie d'attacco XSS.
- **Protezione credenziali con script + pepper:** le password sono salvate con salt univoco, cost factor elevato e pepper lato server, così da rendere non riutilizzabili gli hash anche in caso di compromissione del solo database.
- **Controllo accessi real-time su SSE:** l'apertura dei canali Server-Sent Events è subordinata a validazione JWT; gli stream sono segregati per sessione e ruolo applicativo (cliente/operatore).
- **Mitigazione nativa SQL Injection:** l'accesso dati è concentrato nei repository/adapters PostgreSQL con query parametrizzate, evitando concatenazioni manuali di input utente in statement SQL.

- **Policy CORS restrittiva:** il backend accetta richieste solo da origini autorizzate tramite regole esplicite (incluse validazioni regex per ambienti certificati), riducendo rischi CSRF e chiamate cross-origin abusive.

Queste scelte permettono di proteggere l'intero ciclo operativo, dalla conversazione in chat alla persistenza dell'ordine e alla sua esportazione verso ERP.

5 Architettura logica

Questa sezione descrive la struttura logica di *SmartOrder*, separando i due sottosistemi principali (backend e frontend) e i pattern impiegati per garantire modularità, testabilità ed evoluzione controllata del codice.

5.1 Backend

5.1.1 Esigenze Architettureali

Il backend deve mantenere disaccoppiata la logica applicativa dalle tecnologie esterne (HTTP, database, servizi AI), in modo da preservare la stabilità del dominio al variare dell'infrastruttura e semplificare il collaudo dei servizi.

5.1.2 Soluzione Architettureale

L'organizzazione segue un'**Architettura Esagonale (Ports and Adapters)**:

- **Core:** servizi applicativi e regole di dominio.
- **Ports:** interfacce astratte che definiscono i contratti richiesti dal Core.
- **Adapters:** implementazioni concrete dei Ports e adattatori di ingresso HTTP.

Vista di insieme: il diagramma UML esagonale evidenzia la separazione tra controller (ingresso), servizi di dominio (Core), ports applicative e adapter infrastrutturali (PostgreSQL e OpenAI), con composizione delle dipendenze nel punto di bootstrap (*main.py*).

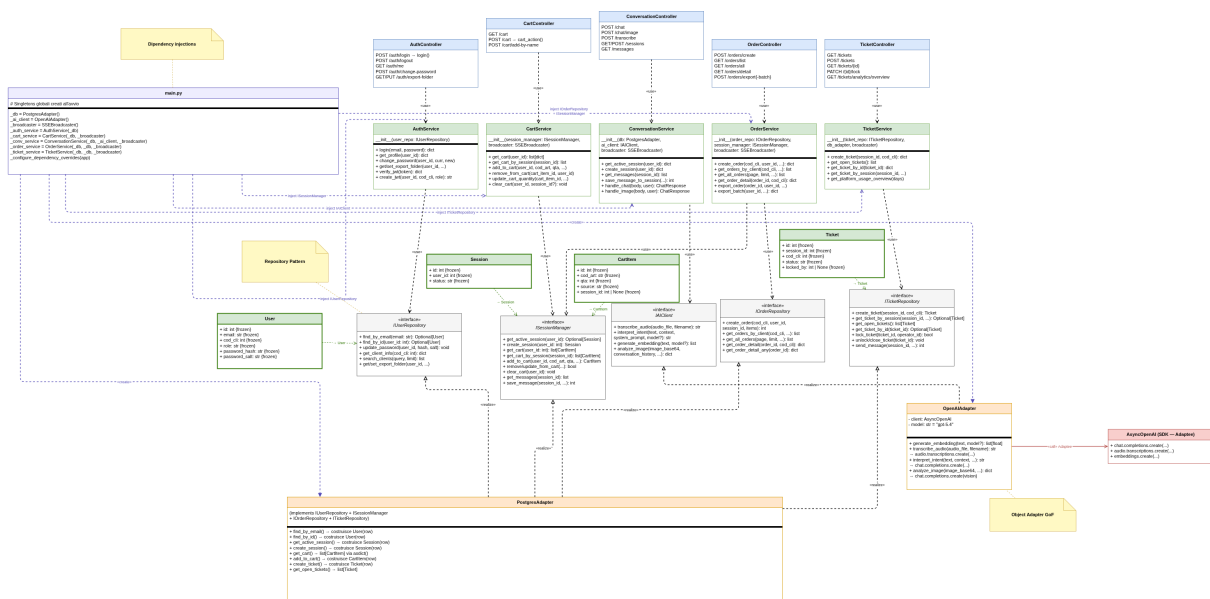


Figura 4: Architettura logica backend secondo il modello esagonale (Ports and Adapters). ([Visualizza a risoluzione piena originale](#))

5.1.3 Realizzazione

Il Core opera operando esclusivamente su interfacce e oggetti di dominio, senza dipendere direttamente dai dettagli dei framework. Per dimostrare l'uso rigoroso del pattern Ports and Adapters, vengono applicati i seguenti principi ai confini del sistema:

- **Adattamento in ingresso:** i controller FastAPI agiscono da *driving adapters*. Ricevute le richieste esterne, il Controller non passa la rappresentazione grezza (es. il payload JSON in arrivo) direttamente al Service. Al contrario, valida e adatta l'input in interfacce di *Use Case* (DTO o oggetti applicativi) immediatamente fruibili dalla logica di business.
- **Adattamento in uscita:** il Service contiene la pura logica di dominio. Quando necessita di interfacciarsi con il database o servizi esterni, passa esclusivamente oggetti di dominio alle porte sottostanti (es. interfacce dei repository), senza dipendere in alcun modo da rappresentazioni esterne, query SQL o librerie tecnologiche. Gli adapter infrastrutturali (es. i driver per PostgreSQL o OpenAI) implementano queste porte ricevendo e convertendo l'oggetto di dominio nel formato tecnico richiesto.

Questa struttura svincola del tutto il Core dalla tecnologia, assicurando un alto grado di testabilità in isolamento.

5.2 Frontend

5.2.1 Esigenze Architettureali

Il frontend richiede una separazione netta tra presentazione, stato applicativo e accesso ai dati, con supporto ad aggiornamenti real-time e interazioni frequenti sul carrello senza degradare la reattività dell'interfaccia.

5.2.2 Soluzione Architettureale e Separazione delle Responsabilità

Il frontend adotta una **Component-Based Architecture** organizzata applicando rigorosamente una scomposizione a livelli delle responsabilità:

- **Presentation Logic:** I componenti puri (*dumb components* all'interno di `components/ui/`) ricevono dati dai livelli superiori e gestiscono il rendering dell'interfaccia utente. Non prendono decisioni di business.
- **View Logic:** Componenti funzionali (*smart components* o pages in `app/`) che formattano i dati, gestiscono lo stato dell'interazione locale e coordinano la renderizzazione.
- **Application Logic (State e Orchestrazione):** Implementata nei custom hook (`hooks/`) e nei context provider (`contexts/` come `CartContext`). Questi orchestrano eventi asincroni e mantengono coerente lo stato locale del sistema.
- **Business Logic / API Abstraction Layer:** Racchiusa all'interno dei file di servizio in `services/` (es. `cartService`, `authService`). Questi agiscono come *Facade*, mappando modelli di dominio ai relativi endpoint.

I pattern principali adottati nel disaccoppiamento di tali logiche includono:

- **Facade:** il layer `services/` incapsula la comunicazione HTTP, isolando i componenti UI dai dettagli di rete.
- **Observer:** Context API e flussi SSE propagano gli aggiornamenti di stato ai componenti sottoscritti.
- **Singleton (module-level):** i servizi API sono condivisi come istanze di modulo, con configurazione centralizzata.

5.2.3 Realizzazione

Per la gestione dello stato globale è stata adottata la **React Context API**, con separazione in due provider principali e un service layer dedicato:

- **SessionContext**: gestisce identità utente (`sessionId`), cronologia dei messaggi e ciclo di vita della sessione; integra l'hook `useSSE` per ricevere aggiornamenti real-time (es. messaggi operatore) e sincronizzarli nello stato condiviso.
- **CartContext**: gestisce lo stato del carrello con strategia *Optimistic UI*; le modifiche alle quantità vengono rese subito in interfaccia e inviate al backend con *debounce* a 600ms, per ridurre burst di chiamate su interazioni rapide.
- **Servizi disaccoppiati**: la comunicazione API è incapsulata in `services/`, così i context restano focalizzati su stato, reattività e sincronizzazione.

Questa soluzione è stata scelta per mantenere un equilibrio tra leggerezza e robustezza: semplicità implementativa coerente con un MVP, esperienza utente fluida grazie agli aggiornamenti ottimistici, propagazione immediata degli eventi server via SSE e maggiore modularità nel testing (componenti dipendenti da sessione e carrello testabili in modo indipendente).

5.2.4 Overview Globale dei Componenti

Il seguente Component Diagram strutturale illustra la mappa delle dipendenze dell'intero applicativo React/Next.js. Esso rende visiva la netta separazione delle responsabilità (Presentation UI, State Contexts, Hooks per l'asincronia e Service Facade per il proxy HTTP/SSE) documentando il flusso unidirezionale delle chiamate dalla View verso i driver backend.

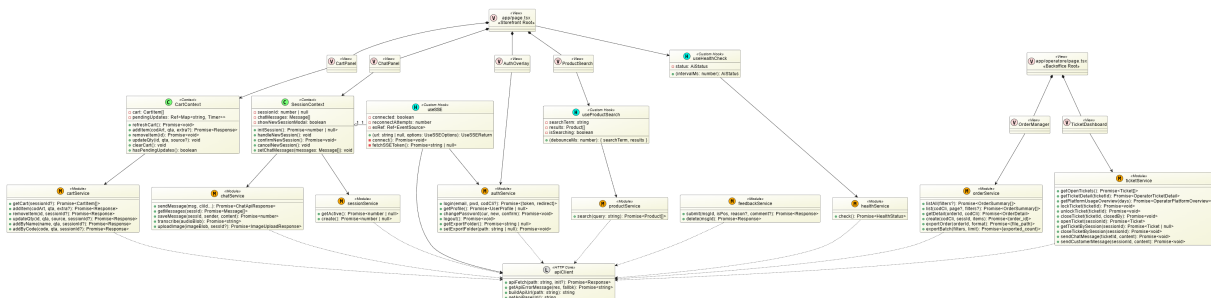


Figura 5: Component Diagram globale dell'architettura logica Frontend. ([Visualizza a risoluzione piena originale](#))

5.3 Design pattern adottati

Questa sottosezione mantiene una vista problem-oriented, separando i pattern effettivamente adottati nel backend e nel frontend.

5.3.1 Pattern backend

5.3.1.1 Dependency Injection

- **Esigenze Architettureali**: I servizi applicativi devono essere testabili e sostituibili senza modificare controller e logica di dominio. Creare dipendenze "in linea" nei componenti avrebbe introdotto accoppiamento forte e inizializzazioni duplicate.

- **Soluzione Architeturale:** Le dipendenze vengono composte nel punto di avvio e iniettate nei componenti tramite provider del framework, mantenendo separati il wiring infrastrutturale e il comportamento di dominio.
- **Realizzazione:** Controller e servizi ricevono istanze già risolte; in collaudo e test si possono iniettare implementazioni alternative (mock/stub) senza intervenire sul codice applicativo.

5.3.1.2 Principi SOLID rispettati L'iniezione delle dipendenze applica il **Dependency Inversion Principle**: i servizi dipendono da astrazioni e non da classi concrete. Inoltre favorisce l'**Open/Closed Principle**, poiché nuove implementazioni possono essere introdotte nel composition root senza modificare la logica di dominio esistente.

Esempio: il composition root in `main.py` costruisce adapter e broadcaster, quindi inietta le dipendenze nei servizi applicativi (`CartService`, `AuthService`, `ConversationService`). Nei test le dipendenze possono essere sostituite (es. `FakeSessionManager`) senza modificare la logica di dominio.

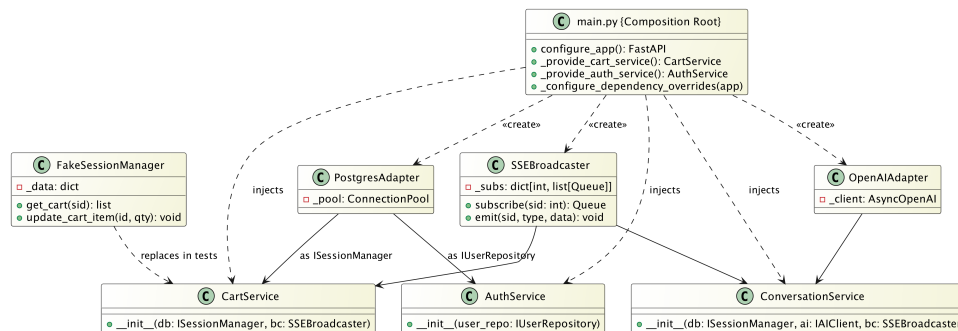


Figura 6: Pattern Dependency Injection: composizione nel bootstrap e iniezione nei servizi applicativi.

5.3.1.3 Repository

- **Esigenze Architeturali:** La logica di business non deve dipendere da query SQL, transazioni e dettagli di schema, per evitare propagazione delle modifiche infrastrutturali nel Core.
- **Soluzione Architeturale:** L'accesso ai dati è mediato da interfacce repository orientate ai casi d'uso (lettura/scrittura ordini, sessioni, ticket), così da mantenere stabile il contratto visto dal dominio.
- **Realizzazione:** I servizi lavorano su repository astratti; gli adapter concreti gestiscono SQL, mapping e vincoli di persistenza, preservando il disaccoppiamento tra dominio e DBMS.

5.3.1.4 Principi SOLID rispettati Il pattern supporta il **Single Responsibility Principle**: la logica applicativa resta distinta dalla persistenza. Rafforza anche il **Dependency Inversion Principle**, perché i service dipendono dal contratto repository; il dettaglio tecnologico (PostgreSQL, mock) è confinato negli adapter concreti.

Esempio: `AuthService` dipende da `IUserRepository` e usa il contratto per login/lookup utenze. L'implementazione può essere `PostgresAdapter` in esercizio o `FakeUserRepository` in test: il servizio mantiene invariata l'interfaccia d'uso e resta indipendente dalla persistenza.

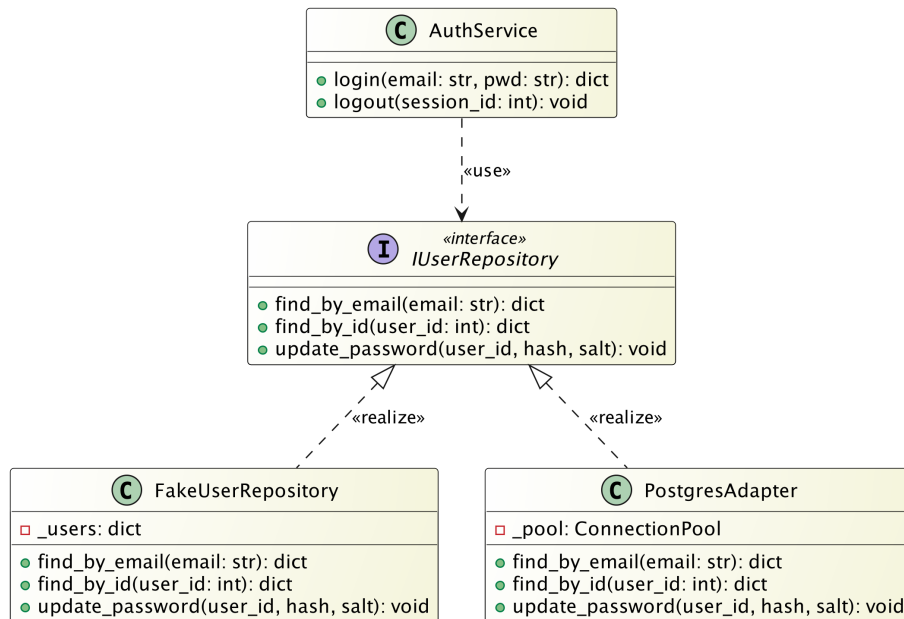


Figura 7: Pattern Repository: AuthService dipende da IUserRepository, con implementazioni sostituibili.

5.3.1.5 Adapter (Ports and Adapters)

- **Esigenze Architettureali:** Il Core deve integrare database e servizi AI senza conoscere SDK, payload e protocolli specifici dei provider.
- **Soluzione Architettureale:** Le Ports definiscono contratti stabili; i driven adapter traducono le operazioni di dominio in chiamate tecniche verso le integrazioni esterne.
- **Realizzazione:** Nel caso AI, ConversationService dipende da IAIClient e non dall'SDK OpenAI. OpenAIAdapter implementa l'interfaccia e converte i metodi applicativi (interpret_intent, transcribe_audio, generate_embedding) nelle chiamate specifiche di AsyncOpenAI.

5.3.1.6 Principi SOLID rispettati L'Adapter concretizza il **Dependency Inversion Principle** mantenendo stabile il contratto verso il dominio. In ottica **Open/Closed Principle**, il sistema puo' integrare nuovi provider o protocolli aggiungendo adapter dedicati senza modificare i servizi core.

Esempio: il diagramma mostra un adattamento *port-to-sdk*: il contratto applicativo resta stabile nel Core, mentre la traduzione dei payload e delle API del provider e' confinata nell'adapter. In questo modo, un cambio di provider AI impatta il layer di integrazione senza modificare il servizio di dominio.

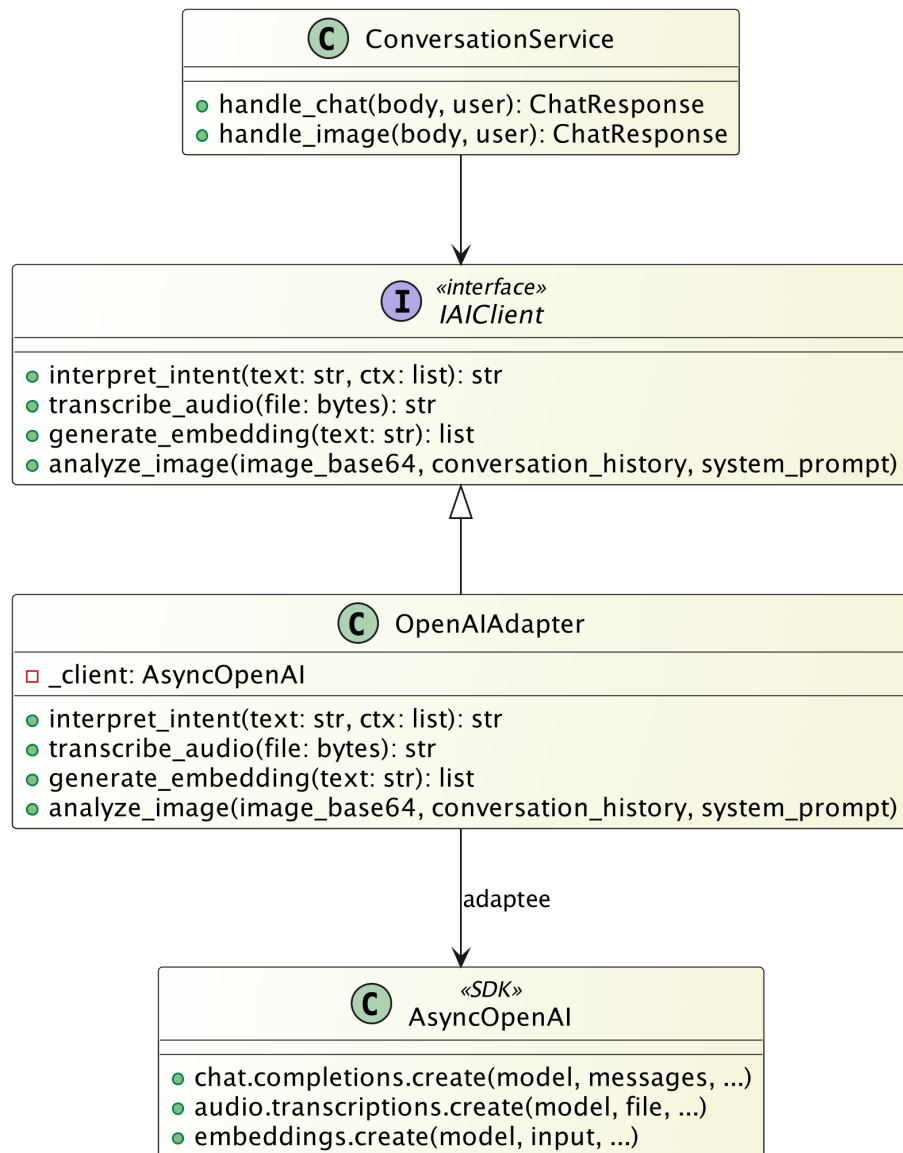


Figura 8: Pattern Adapter applicato all'integrazione AI tramite **OpenAIAdapter**.

5.3.2 Pattern frontend

5.3.2.1 Facade (Service Layer frontend)

- **Esigenze Architettureali:** La UI deve restare disaccoppiata da protocollo HTTP, costruzione URL, header di sicurezza, cookie CSRF e gestione degli errori applicativi.
- **Soluzione Architettureale:** Il Service Layer in `services/` espone servizi specialistici (es. `cartService`, `authService`, `orderService`) che agiscono da Facade verso le API backend.
- **Realizzazione:** I componenti React invocano operazioni di dominio ad alto livello; internamente i servizi orchestrano utilità di basso livello (`apiFetch`, `buildApiUrl`, `getApiBaseUrl`, `getApiErrorMessage`) e uniformano il trattamento delle risposte HTTP non valide.

5.3.2.2 Principi SOLID rispettati La Facade favorisce il **Single Responsibility Principle**: i componenti UI gestiscono presentazione e stato locale, mentre il layer servizi centralizza

la comunicazione remota. L'approccio rispetta anche l'**Interface Segregation Principle**, esponendo API frontend mirate per casi d'uso specifici.

Caso d'uso (cartService): operazioni come `addItem`, `removeItem` e `updateQty` coordinano routing endpoint, serializzazione parametri (es. `cod_art`, `quantity`) e invocazione rete. L'incapsulamento riduce la superficie di modifica: variazioni degli endpoint o dello schema di risposta sono localizzate nel solo Service Layer.

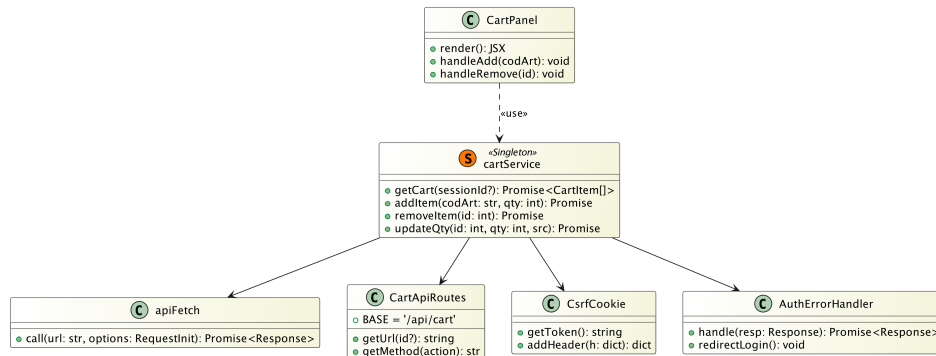


Figura 9: Pattern Facade nel frontend: `cartService` come interfaccia semplificata tra UI e client API.

5.3.2.3 Observer (SSE broadcaster + observer client)

- **Esigenze Architettureali:** Gli aggiornamenti real-time di sessione (es. carrello) devono essere propagati a piu' consumer in modo coerente e senza polling.
- **Soluzione Architettureale:** Il subject e' `SSEBroadcaster`, che mantiene i subscriber per sessione; gli observer implementano `IObserver` e ricevono eventi tramite `update(event)`.
- **Realizzazione:** L'implementazione adottata e' unica e coincide con il diagramma: `CartService` emette eventi verso `SSEBroadcaster`, mentre `SSEClientQueue` materializza la coda asincrona del singolo subscriber SSE.

5.3.2.4 Principi SOLID rispettati Il pattern rispetta l'**Open/Closed Principle**: nuovi observer possono essere introdotti senza modificare il subject. Inoltre supporta il **Dependency Inversion Principle**, perche' il broadcaster interagisce tramite contratto `IObserver` anziche' dipendere da consumer concreti.

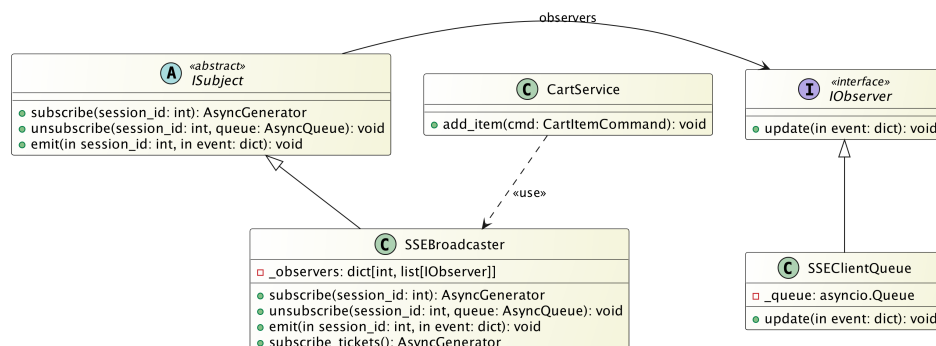


Figura 10: Pattern Observer per propagazione eventi real-time via SSE.

5.3.2.5 Singleton (module-level services)

- **Esigenze Architettureali:** Le chiamate API devono mantenere configurazione uniforme ed evitare duplicazioni di client e logica di rete.
- **Soluzione Architettureale:** I servizi sono esportati come istanze condivise a livello modulo, con configurazione centralizzata del client HTTP.
- **Realizzazione:** L'interfaccia utilizza una singola fonte di configurazione per autenticazione, gestione errori e politiche di chiamata, migliorando coerenza e manutenibilit .

5.3.2.6 Principi SOLID rispettati L'uso controllato del Singleton a livello modulo mantiene una responsabilit  unica di configurazione e ciclo vita del client (**Single Responsibility Principle**). I componenti consumatori restano disaccoppiati dal dettaglio di inizializzazione e dipendono da un servizio gi  configurato, coerente con il **Dependency Inversion Principle** applicativo.

Caso d'uso (cartService + CartProvider): il modulo `cartService` e' esportato come istanza condivisa e centralizza la comunicazione HTTP con il backend. In parallelo, `CartProvider` gestisce l'unica sorgente di verita' dello stato carrello per la sessione utente: la funzione `updateQty` applica aggiornamento ottimistico locale, annulla eventuali timer gi  attivi per lo stesso prodotto e persiste la modifica dopo una finestra di debounce di 600 ms. In caso di errore remoto, il provider riallinea lo stato ricaricando i dati reali dal server.

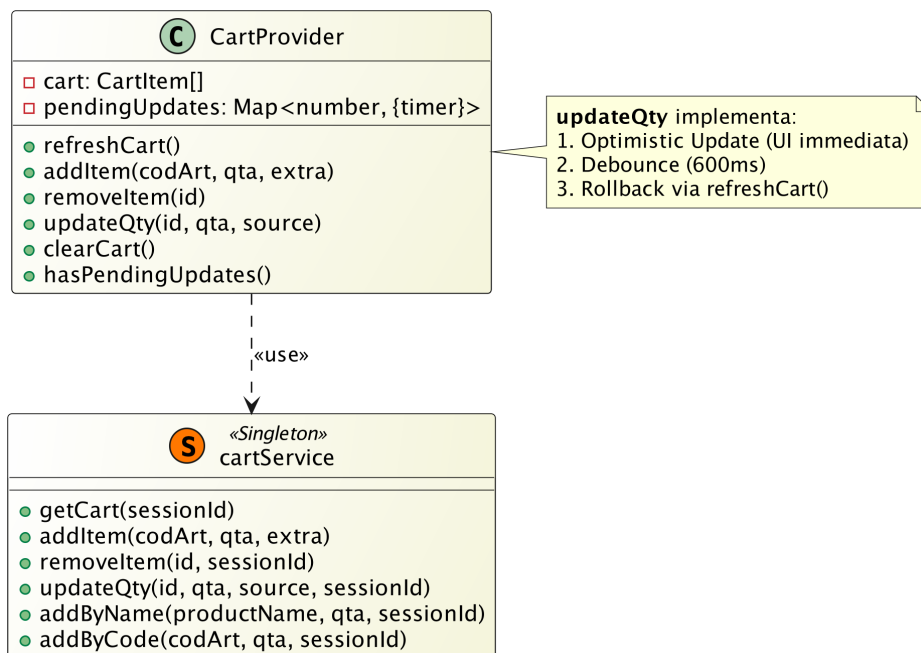


Figura 11: Pattern Singleton nel frontend: `cartService` come servizio condiviso e `CartProvider` come sorgente unica dello stato carrello.

5.3.2.7 Strategie applicative: Optimistic UI e Debounce

- **Esigenze Architettureali:** Le operazioni frequenti sul carrello devono restare reattive lato utente, ma senza generare picchi di richieste concorrenti verso il backend.
- **Soluzione Architettureale:** Lo stato locale viene aggiornato in modo ottimistico e le chiamate ripetute in rapida sequenza vengono aggregate tramite debounce.

- **Realizzazione:** Le modifiche sono visibili subito in UI; in caso di errore viene eseguito rollback/sincronizzazione, mentre gli update ravvicinati vengono consolidati in una richiesta finale.

6 Database

Questa sezione descrive l’architettura dei dati del sistema *SmartOrder*, definendo il ruolo del database, le macro-aree informative gestite, i vincoli di integrità applicati e le modalità di interscambio dati con il sistema gestionale esterno.

6.1 Ruolo del database nel sistema

Il sistema si avvale di **PostgreSQL** (in hosting cloud su Railway) come unica fonte dati transazionale (Single Source of Truth) per il backend FastAPI. Il database relazionale assolve a tre ruoli fondamentali:

- **Persistenza Transazionale:** memorizza in modo consistente lo stato delle sessioni, dei carrelli in lavorazione, dei ticket di assistenza e lo storico degli ordini consolidati.
- **Replica del Catalogo ERP:** ospita le tabelle anagrafiche dei prodotti e dei clienti sincronizzate dal gestionale aziendale, permettendo al sistema di operare con logiche di business locali (es. verifica assortimenti).
- **Motore di Ricerca Vettoriale:** tramite l’estensione **pgvector**, il database archivia gli *embeddings* generati dai modelli LLM e risolve le query semantiche di "Vector Search", mappando le descrizioni in linguaggio naturale ai codici articolo reali.

6.2 Macro-aree informative

Il modello relazionale del database è logicamente suddiviso in macro-aree, ciascuna responsabile di un dominio specifico dell’applicazione.

Area Logica	Contenuto principale e Responsabilità
Identità e Accesso	Entità relative agli utenti, ruoli (Cliente, Operatore), credenziali crittografate (hash scrypt) e gestione dei token/sessioni attive.
Catalogo e Assortimento	Anagrafica articoli (anaart), metadati descrittivi, <i>embeddings</i> vettoriali per la ricerca semantica e tabelle di visibilità/assortimento per singolo cliente (asscli).
Carrello e Ordini	Record transazionali che rappresentano il carrello corrente della sessione, le singole righe prodotto con le relative quantità e lo storico degli ordini finalizzati pronti per la consultazione (Storico Ordini).
Ticketing e Chat	Entità per la gestione dell’assistenza (<i>Human-in-the-loop</i>). Include i record dei Ticket (stati: "Aperto", "In lavorazione", "Chiuso"), l’assegnazione all’Operatore (Lock) e il tracciato cronologico (Log) dei messaggi di chat scambiati.
Log AI e Ottimizzazione	Archiviazione diagnostica delle interazioni con il modello LLM. Comprende il <i>Confidence Score</i> , i feedback degli utenti utili ad un possibile <i>retraining</i> .

Tabella 13: Aree logiche del modello dati relazionale.

6.3 Struttura delle principali tabelle

In base allo schema SQL in uso, di seguito è riportata la struttura e il significato dei campi più rilevanti per le entità centrali del sistema.

6.3.0.1 Anagrafica Articoli (**anaart**)

Tabella fondamentale per la gestione del catalogo ordini:

- **cod_art**: codice articolo (Primary Key).
- **des_art**: descrizione estesa del prodotto.
- **stato**: stato logico di disponibilità del prodotto.
- **linea**: indica il settore reale a cui appartiene il prodotto (es. acqua, vino, ecc.).
- **famiglia**: indica la fascia o qualità del prodotto (es. premium, standard, ecc.).
- **des_um**: unità di misura base di vendita (es. casse, cartone).
- **des_tipo_um**: unità di misura dei singoli prodotti all'interno della singola confezione (es. pezzi, bottiglie).
- **pezzi_conf**: quantità calcolata in **des_tipo_um** situata all'interno della misura base **des_um**.
- **peso_netto_conf**: peso netto per singola unità di **des_um**.

Nota operativa: L'attributo nativo **settore** presente a schema non contiene in realtà i valori di business indicanti il settore del prodotto, i quali si trovano invece nel campo **linea**.

6.3.0.2 Sessioni e Chat (**chat_sessions**, **chat_messages**)

Il nucleo conversazionale salva lo stato delle interazioni in maniera persistente:

- **status**: indica se la sessione è *active*, *completed* o *archived*.
- **sender** (**chat_messages**): traccia l'emittente esatto del messaggio tra *user*, *ai*, *operator* e *system*.
- **image_data** e **ocr_text**: permettono l'invio di ordini multimodali memorizzando un'immagine decodificata in base64 e l'interpretazione testuale del servizio LLM associato.

6.3.0.3 Carrello (**cart_items**)

Le righe del carrello in elaborazione prevedono una cronologia di modifica e confidenza:

- **cod_art** e **qta**: riferimento all'articolo in assortimento e alle quantità richieste dall'utente.
- **source** e **last_updated_by**: attribuiscono l'inserimento/modifica iniziale e finale a un utente, all'intelligenza artificiale o a un operatore.
- **ai_confidence**: livello di certezza estrapolato dal LLM (fondamentale per far scattare controlli automatizzati o trigger per il backoffice in caso di bassi valori).

6.3.0.4 Ordini Finalizzati (**orders**, **order_items**)

Rappresentano la formalizzazione del carrello al netto delle modifiche di *Human-in-the-Loop*. Gli ordini mantengono un tracciamento dei campi **source** per finalità di audit. Troviamo inoltre il flag di stato **esportato** per gestire l'avvenuta sincronizzazione JSON.

6.3.0.5 Identità Temporanee e Persistenti (`app_users`, `anaccli`, `asscli`)

- `app_users`: include il ruolo autenticato (*customer* o *admin*), il collegamento al cliente via `cod_cli`, e soprattutto il campo `export_folder` che specifica il path assoluto della cartella di rete dove l'operatore desidera ricevere i file JSON degli ordini finalizzati. Questo path è configurabile per operatore ed è fondamentale per l'integrazione ERP distribuita.
- `anaccli`: generalità e ragione sociale del cliente.
- `asscli`: tabella ponte di assortimento (collega `cod_cli` e `cod_art`). Esclusivamente quanto è qui presente risulta ordinabile dal Cliente interessato.

6.3.0.6 Gestione Ticket e Lock Operatore (`tickets`) Gestisce la supervisione manuale di casi a bassa confidence AI o anomalie:

- `status`: stati discreti ("aperto", "in_lavorazione", "chiuso") con vincoli *CHECK*.
- `locked_by`: ID dell'operatore che ha preso in carico il ticket. Implementa un meccanismo di pessimistic locking per prevenire race condition: un operatore non può modificare un ticket già bloccato da un altro operatore.
- `created_at`, `updated_at`: timestamp per audit trail e metriche di SLA.

6.3.0.7 Feedback e Ottimizzazione LLM (`message_feedbacks`) Registra il feedback implicito ed esplicito sui messaggi AI generati:

- `is_positive`: valore booleano che indica se l'utente ha trovato utile il messaggio.
- `reason_category`: categorizzazione del feedback (es. "errato", "incomplete", "inappropriata").
- `comment`: note testuali dell'utente. Questi dati alimentano il processo di retraining e fine-tuning del modello LLM.
- Vincolo *UNIQUE*(`message_id`, `user_id`): ogni utente può fornire un solo feedback per messaggio, evitando duplicati.

6.3.0.8 Embedding Vettoriali per Ricerca Semantica (`product_embeddings`) Archivia gli embedding generati dai modelli LLM per abilitare la vector search:

- `embedding`: vettore multidimensionale memorizzato tramite l'estensione PostgreSQL `pgvector`. Ciascun embedding rappresenta la "semantica" testuale dell'articolo (descrizione, linea, famiglia).
- `created_at`: timestamp di generazione dell'embedding (utile per invalidazione e rigenerazione periodica).
- Utilizzo: quando l'utente descrive un prodotto in linguaggio naturale, il sistema genera un embedding della query e cerca i *nearest neighbors* nello spazio vettoriale per recuperare articoli semanticamente simili, anche se con descrizioni non letteralmente corrispondenti.

6.4 Uso dei log per analisi operativa e miglioramento

I dati persistiti nel database non sono utilizzati solo per finalità transazionali, ma potranno anche essere usate per monitoraggio, audit e miglioramento continuo del sistema:

- **Cronologia messaggi (chat_messages):** consente di ricostruire il contesto conversazionale end-to-end, fondamentale per comprendere richieste ellittiche e analizzare casi anomali in fase di debug.
- **Metadati di confidenza (metadata dei messaggi AI):** permettono di valutare come il modello ha selezionato articoli e con quale affidabilità, supportando analisi causale degli errori di suggerimento.
- **Feedback utenti (message_feedbacks):** rendono misurabile la qualità percepita delle risposte AI e costituiscono un dataset supervisionato per prioritizzare ottimizzazioni future (es. RLHF).
- **Stato carrello e ordini (cart_items, orders, order_items):** abilita audit trail operativo su chi ha modificato cosa e quando, utile per verifica di conformità.
- **Ticket di intervento (tickets):** forniscono indicatori sui limiti dell'automazione (frequenza takeover, tempi di gestione), utili per orientare backlog tecnico e miglioramenti di prodotto.

In prospettiva, la combinazione di questi log permette di definire KPI tecnici e di processo (precisione suggerimenti, tempo medio takeover, tasso correzioni manuali) utili per guidare decisioni di evoluzione architetturale.

6.5 Strategie di indicizzazione e performance

Per garantire responsiveness della ricerca e delle operazioni transazionali, il schema implementa indici composti e strategici:

- **Indici Compositi su Sessioni e Chat:** `idx_chat_sessions_user_status` (`user_id`, `status`) accelera il retrieval delle sessioni attive di un utente. `idx_chat_messages_session_id` ottimizza il caricamento della cronologia.
- **Indici su Feedback:** `idx_message_feedbacks_message_user_created` permette query efficienti sul feedback per singolo messaggio, utente e timestamp, fondamentali per il report dei contributi.
- **Vincolo Unico Composito:** `uq_cart_session_product` previene inserimento duplicato di uno stesso articolo in un carrello nella medesima sessione.
- **Indice Vettoriale:** `idx_product_embeddings_cosine` utilizza l'algoritmo IVFFlat (Inverted File Flat) con 100 liste per la ricerca vettoriale efficiente su high-dimensional space. L'operatore di distanza è la "cosine distance", appropriato per embedding semantici normalizzati.

6.6 Vincoli dati rilevanti

Per garantire la robustezza del sistema e prevenire l'invio di ordini commercialmente invalidi al sistema ERP, il database impone rigorosi vincoli di integrità e di dominio:

- **Vincolo di Assortimento (Foreign Key logica):** un prodotto può essere inserito nel carrello di una sessione solo se esiste una corrispondenza valida nella tabella degli assortimenti autorizzati (`asscli`) per il Cliente in esame.

- **Vincolo di Disponibilità:** controlli a livello database (*anaart*) impediscono l'immissione in ordine di articoli marcati con stato "sospeso" o "non disponibile".
- **Integrità Referenziale Standard:** chiavi esterne (Foreign Keys) con regole di propagazione *ON DELETE CASCADE/RICT* collegano indissolubilmente le righe dell'ordine all'ID Ordine padre e alla relativa anagrafica Cliente, prevenendo record orfani.
- **Vincolo di Quantità Commerciale:** i campi quantitativi delle righe ordine possiedono vincoli (*CHECK constraints*) per impedire valori nulli, negativi o incompatibili con i multipli di vendita (es. logica a "casse").

6.7 Output verso ERP

Il database di SmartOrder funge da ambiente di *staging* e composizione. Al termine del processo di checkout, i dati transazionali (intestazione ordine, righe, quantità e ID cliente) estratti dalle tabelle relazionali subiscono un processo di serializzazione.

Il sistema genera un **payload strutturato in formato JSON**. Questo file viene immediatamente depositato in una cartella condivisa nel computer, che rappresenta il reale punto di aggancio e consegna formale verso il sistema ERP aziendale.

6.7.1 Struttura del payload JSON esportato

L'output è un oggetto completo che include testata ordine, dettaglio righe e contesto conversazionale. Una forma semplificata del tracciato è la seguente:

Listing 1: Struttura logica del JSON esportato verso ERP.

```

1 {
2   "order_id": "ORD-2026-000123",
3   "cod_cli": "CLI0456",
4   "data_ord": "2026-04-14T10:22:31Z",
5   "session_id": "sess_7f8b9c",
6   "items": [
7     {
8       "cod_art": "A001",
9       "qta_ordinata": 4,
10      "des_art": "Acqua Naturale 1L",
11      "des_um": "casse",
12      "linea": "beverage",
13      "famiglia": "standard",
14      "source": "ai",
15      "ai_confidence": 0.93
16    }
17  ],
18  "messages": [
19    {
20      "sender": "user",
21      "content": "Aggiungi altre 2 casse",
22      "feedback": null
23    }
24  ]
25 }
```

Campi principali:

- **Testata ordine:** *order_id*, *cod_cli*, *data_ord*, *session_id* identificano in modo univoco documento, cliente e contesto operativo.

- **Dettaglio articoli (items):** ogni riga include identificativi ERP (`cod_art`), quantità (`qta_ordinata`), metadati descrittivi e attributi di provenienza/affidabilità (`source`, `ai_confidence`).
- **Contesto conversazionale (messages):** storico dei messaggi e dei feedback associati, utile per tracciabilità delle decisioni e verifica ex-post.

Motivazioni architetturali (Enterprise Ready):

- **Integrazione standard:** JSON e' interoperabile con ERP moderni e processi ETL/ELT senza conversioni proprietarie.
- **Mappatura diretta:** l'uso di `cod_cli` e `cod_art` consente caricamento deterministico verso anagrafiche gestionali esistenti.
- **Automazione batch:** la scrittura in cartella condivisa permette import incrementale da parte del gestionale senza inserimento manuale.
- **Audit e compliance:** campi `source`, `ai_confidence` e `messages` rendono spiegabile la formazione dell'ordine.
- **Separazione delle responsabilita':** SmartOrder normalizza e valida il dato; l'ERP mantiene i processi amministrativi e logistici a valle.

Nota architetturale - Disaccoppiamento tramite Adapter: In conformità al pattern Ports and Adapters, il Core applicativo di SmartOrder **non scrive mai direttamente** nel database dell'ERP. Invece, i servizi di dominio delegano la responsabilità dell'output a un **driven adapter specializzato** (Export Adapter). Questo adapter:

- Riceve dal Core l'ordine strutturato come entità di dominio (pura logica di business).
- Trasforma l'entità in un payload JSON conforme al formato atteso dal sistema gestionale.
- Scrive il file nella cartella condivisa del File System.

Questo disaccoppiamento consente evoluzione indipendente: se domani il sistema ERP cambia formato o destinazione (es. database diretti, API, message queue), basterà sostituire o estendere l'adapter senza toccare la logica di dominio o i servizi di business.

7 Architettura di dettaglio

In questa sezione vengono descritti gli elementi di dettaglio architetturale del sistema, suddivisi per ambito backend e frontend. Per ciascun componente/classe/interfaccia sono riportati:

- Titolo dell'elemento;
- Descrizione della responsabilità;
- Descrizione di attributi e metodi;
- Diagramma UML dedicato.

7.1 Backend - C4 Livello 4: Classi e interfacce

La sottosezione presenta classi e interfacce backend identificate nel C4 Livello 4.

7.1.1 Controllers (Inbound Ports)

7.1.1.1 AuthController **Descrizione della classe:** gestisce gli endpoint di autenticazione, cambio password, gestione cartella di esportazione e rilascio token SSE, traducendo le richieste HTTP in invocazioni ai casi d'uso applicativi.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `login(body: LoginRequest)`: avvia autenticazione utente e genera la sessione applicativa tramite cookie JWT.
- `logout()`: invalida la sessione autenticata corrente cancellando i cookie.
- `me()`: restituisce i dati dell'utente correntemente autenticato.
- `change_password(body: ChangePasswordRequest)`: gestisce il flusso di aggiornamento credenziali.
- `get_export_folder()`: recupera la cartella di esportazione configurata dall'operatore.
- `set_export_folder(body)`: imposta o aggiorna la cartella di esportazione dell'operatore.
- `sse_token()`: restituisce il JWT dal cookie HTTPOnly per connessioni SSE dirette.

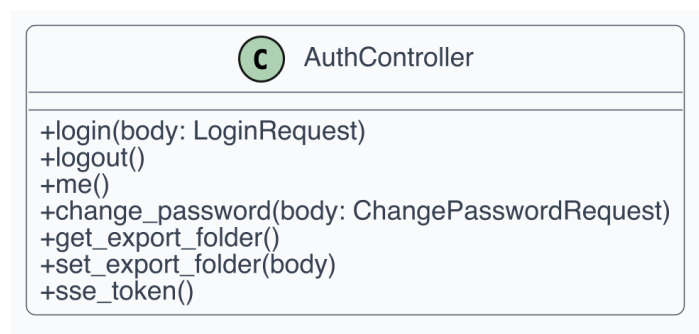


Figura 12: UML della classe `AuthController`.

7.1.1.2 ConversationController **Descrizione della classe:** espone gli endpoint per la conversazione multimodale (testo, audio, immagini) e il recupero dello storico chat e sessioni.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `chat(body: ChatRequest)`: elabora input testuale inviato dall'utente tramite la pipeline AI.
- `chat_image(file: UploadFile)`: gestisce input multimodale includendo immagini con OCR via GPT-5.4 Vision.
- `save_message_to_session(body: SaveMessageRequest)`: salva un messaggio UI nella sessione per persistenza.
- `transcribe(file: UploadFile)`: effettua trascrizione di messaggi vocali via Whisper.
- `get_active_session()`: recupera la sessione conversazionale attiva dell'utente.
- `create_session()`: istanzia una nuova sessione conversazionale.
- `get_messages(session_id: int)`: recupera lo storico messaggi della sessione.

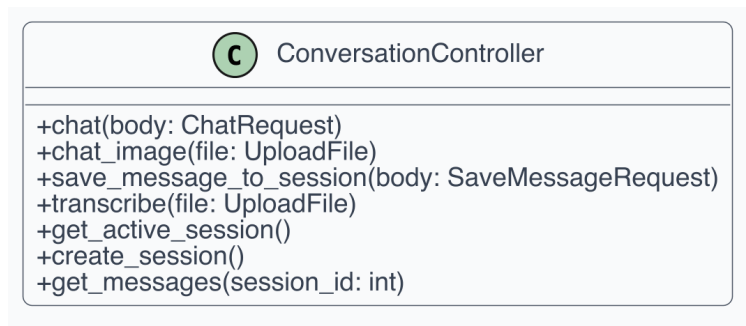


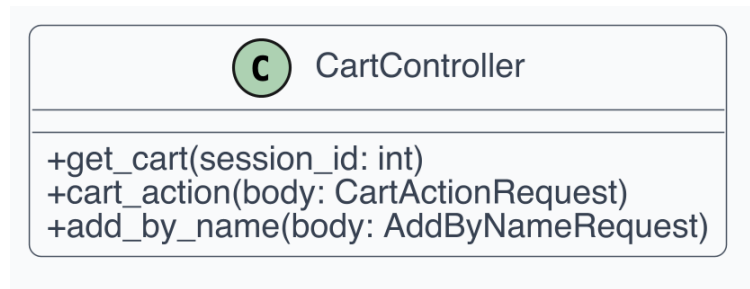
Figura 13: UML della classe `ConversationController`.

7.1.1.3 CartController **Descrizione della classe:** governa le operazioni sul carrello (inserimento, modifica, rimozione) e l'aggiunta per nome prodotto.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `get_cart(session_id: int)`: restituisce l'elenco corrente delle righe prodotto nel carrello.
- `cart_action(body: CartActionRequest)`: agisce centralmente per inserire, modificare quantità o rimuovere articoli dal carrello.
- `add_by_name(body: AddByNameRequest)`: aggiunge un prodotto al carrello ricercandolo per nome testuale.

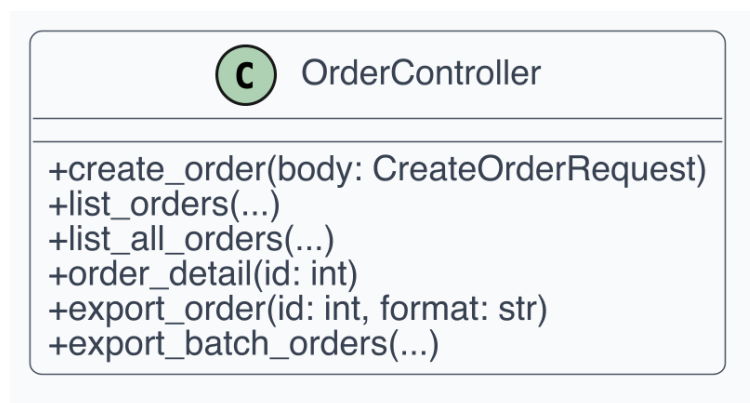
Figura 14: UML della classe **CartController**.

7.1.1.4 OrderController **Descrizione della classe:** espone le API per creazione ordini, consultazione storico, dettaglio singolo ordine, esportazione singola e batch.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `create_order(body: CreateOrderRequest)`: tramuta un carrello finale in ordine persistente.
- `list_orders(...)`: recupera lo storico ordini paginato per il cliente loggato.
- `list_all_orders(...)`: interfaccia estesa a disposizione dell'operatore con filtri avanzati.
- `order_detail(id: int)`: recupera il dettaglio completo di un ordine specifico.
- `export_order(id: int, format: str)`: esportazione dell'ordine singolo in formato JSON o CSV.
- `export_batch_orders(...)`: avvia macroesportazione filtrata di ordini multipli.

Figura 15: UML della classe **OrderController**.

7.1.1.5 TicketController **Descrizione della classe:** gestisce l'apertura, la presa in carico, la chiusura e la messaggistica dei ticket di assistenza Human-in-the-Loop, sia lato operatore che lato cliente.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `list_open_tickets()`: restituisce la lista dei ticket pendenti (operatori).
- `create_ticket(body: CreateTicketRequest)`: crea un ticket di assistenza per la sessione corrente.
- `get_ticket_by_session(session_id: int)`: ritorna il ticket associato a una sessione.
- `get_analytics_overview(days: int)`: metriche aggregate per la dashboard operatore.
- `get_ticket(ticket_id: int)`: dettaglio di un ticket specifico.
- `lock_ticket(ticket_id: int)`: assegna il ticket all'operatore corrente (lock).
- `send_ticket_message(ticket_id: int, body: SendMessageRequest)`: invia un messaggio operatore nella chat del ticket.
- `unlock_ticket(ticket_id: int)`: rilascia il lock sull'operatore.
- `close_ticket(ticket_id: int)`: chiude un ticket (operatore o cliente).
- `close_ticket_by_session(session_id: int)`: chiude il ticket associato a una sessione (cliente).
- `send_customer_message(session_id: int, body: SendMessageRequest)`: invia un messaggio del cliente nella chat del ticket.

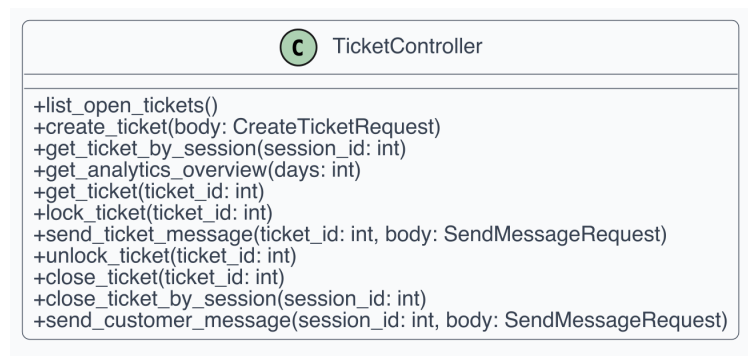


Figura 16: UML della classe `TicketController`.

7.1.1.6 FeedbackController **Descrizione della classe:** espone le API di raccolta e cancellazione feedback relativi alle risposte dell'AI.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `handle_feedback(body: FeedbackRequest)`: salva, aggiorna o cancella il feedback qualitativo relativo alle scelte deduttive dell'LLM.

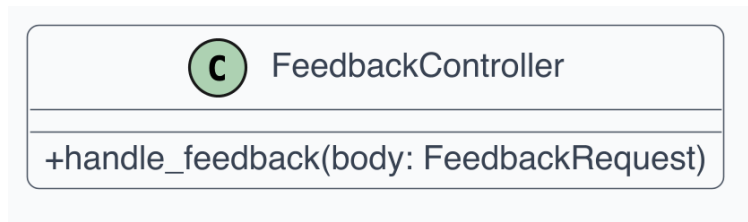


Figura 17: UML della classe **FeedbackController**.

7.1.1.7 ClientController **Descrizione della classe:** espone l’endpoint di ricerca clienti per ragione sociale o codice cliente.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- **search_clients(q: str):** cerca clienti nel database per ragione sociale o codice.

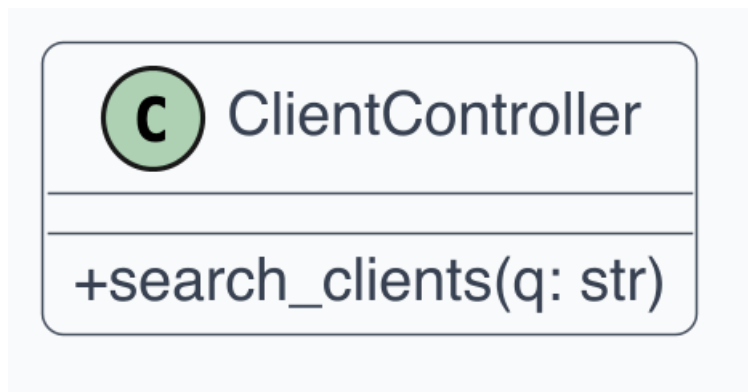


Figura 18: UML della classe **ClientController**.

7.1.1.8 HealthController **Descrizione della classe:** fornisce un endpoint per l’health check e il monitoraggio dello stato di salute del backend.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- **health_check():** verifica lo stato operativo del backend e dei servizi AI collegati.



Figura 19: UML della classe `HealthController`.

7.1.1.9 ProductController **Descrizione della classe:** espone l'endpoint di ricerca prodotti nel catalogo, filtrato per assortimento cliente.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `search_products(q: str)`: cerca prodotti nel catalogo filtrando per l'assortimento del cliente autenticato.



Figura 20: UML della classe `ProductController`.

7.1.1.10 SSEController **Descrizione della classe:** gestisce le connessioni Server-Sent Events per le notifiche in streaming verso client e operatori.

Descrizione degli attributi: nessun attributo di stato esplicitato nel diagramma (controller stateless).

Descrizione dei metodi:

- `sse_tickets()`: stream SSE per aggiornamenti lista ticket (canale operatori).

- `sse_session(session_id: int)`: stream SSE per aggiornamenti real-time su una sessione specifica.

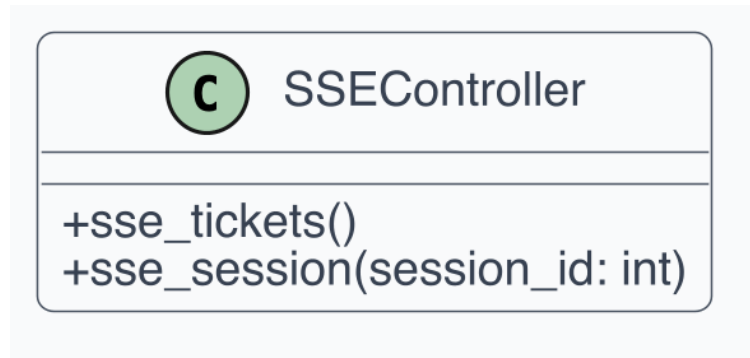


Figura 21: UML della classe SSEController.

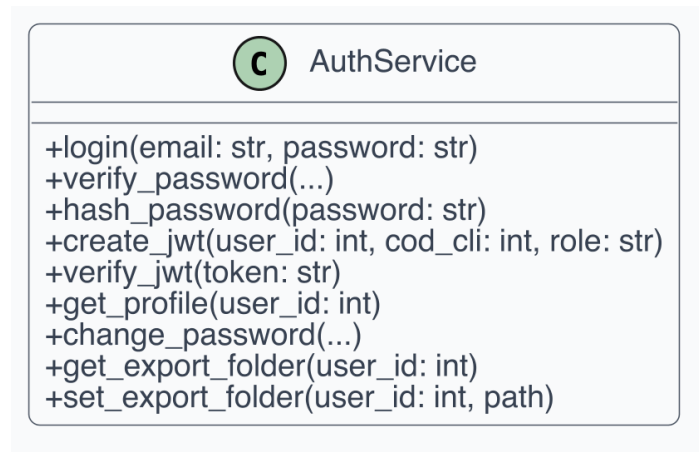
7.1.2 Application Services (Use Cases)

7.1.2.1 AuthService **Descrizione della classe:** incapsula i casi d'uso di autenticazione, gestione credenziali, generazione/verifica JWT e gestione profilo utente.

Descrizione degli attributi: non sono esplicitati attributi nel diagramma; il servizio è modellato come componente applicativo stateless.

Descrizione dei metodi:

- `login(email: str, password: str)`: autentica utente. Ritorna il token e dettagli sessione.
- `verify_password(...)`: gestisce l'hashing e la validazione script multi-profilo.
- `hash_password(password: str)`: genera hash e salt per una nuova password.
- `create_jwt(user_id: int, cod_cli: int, role: str)`: crea un token JWT firmato.
- `verify_jwt(token: str)`: verifica e decodifica un JWT. Ritorna il payload o None.
- `get_profile(user_id: int)`: recupera profilo utente specifico.
- `change_password(...)`: sostituisce l'impronta della password nel database.
- `get_export_folder(user_id: int)`: recupera la cartella di esportazione.
- `set_export_folder(user_id: int, path)`: salva la cartella di esportazione.

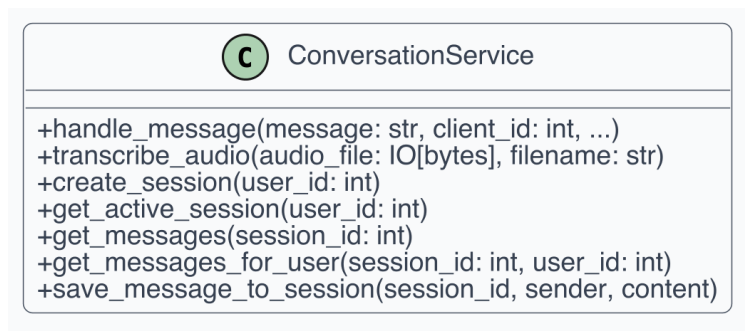
Figura 22: UML della classe **AuthService**.

7.1.2.2 ConversationService **Descrizione della classe:** implementa il flusso conversazionale (testo e audio), la gestione del contesto dialogico e il coordinamento con la pipeline AI.

Descrizione degli attributi: non sono esplicitati attributi di stato nel diagramma.

Descrizione dei metodi:

- `handle_message(message: str, client_id: int, ...)`: orchestra la pipeline decisionale LLM tramite intent recognition.
- `transcribe_audio(audio_file: IO[bytes], filename: str)`: delega trascrizione al servizio AI.
- `create_session(user_id: int)`: crea una nuova sessione conversazionale.
- `get_active_session(user_id: int)`: recupera la sessione attiva per l'utente.
- `get_messages(session_id: int)`: recupera lo storico messaggi della sessione.
- `get_messages_for_user(session_id: int, user_id: int)`: recupera i messaggi filtrati per utente.
- `save_message_to_session(session_id, sender, content)`: salva un messaggio nella sessione.

Figura 23: UML della classe **ConversationService**.

7.1.2.3 CartService **Descrizione della classe:** concentra la logica di business del carrello con supporto per operazioni per utente e per sessione, e notifica SSE sugli aggiornamenti.

Descrizione degli attributi: non sono indicati attributi persistenti nel diagramma.

Descrizione dei metodi:

- `get_cart(user_id: int)`: recupera il carrello dell'utente.
- `get_cart_by_session(session_id: int)`: recupera il carrello della sessione.
- `add_to_cart(user_id, cod_art, qta, ...)`: inserisce un articolo nel carrello con validazione.
- `remove_from_cart(cart_item_id, user_id, ...)`: elimina un articolo dal carrello.
- `update_cart_quantity(cart_item_id, user_id, qta, ...)`: aggiorna la quantità di un articolo.
- `clear_cart(user_id, session_id)`: svuota il carrello dell'utente.

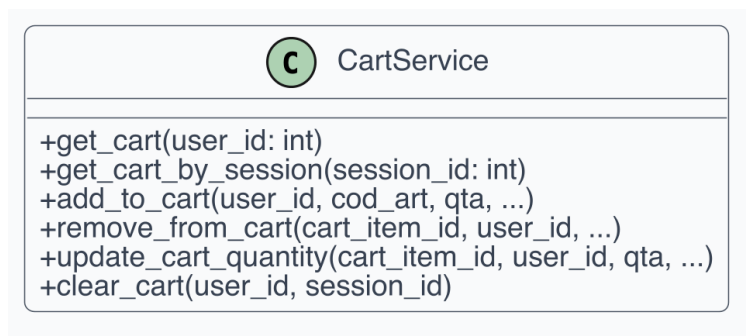


Figura 24: UML della classe `CartService`.

7.1.2.4 OrderService **Descrizione della classe:** fornisce i casi d'uso di creazione ordini, consultazione storico, dettaglio ed esportazione ordini.

Descrizione degli attributi: nessun attributo esplicitato nel diagramma.

Descrizione dei metodi:

- `create_order(cod_cli, user_id, session_id, items)`: crea un ordine validando gli articoli e chiudendo la sessione.
- `get_orders_by_client(cod_cli, page, limit, ...)`: recupera lo storico paginato per cliente.
- `get_all_orders(page, limit, ...)`: dashboard estesa con filtri avanzati per operatori.
- `get_order_detail(order_id, cod_cli)`: vista profonda per singolo ordine.
- `export_order(order_id, user_id, format)`: esporta un ordine in JSON o CSV.
- `export_batch(user_id, ...)`: esportazione batch di ordini filtrati.

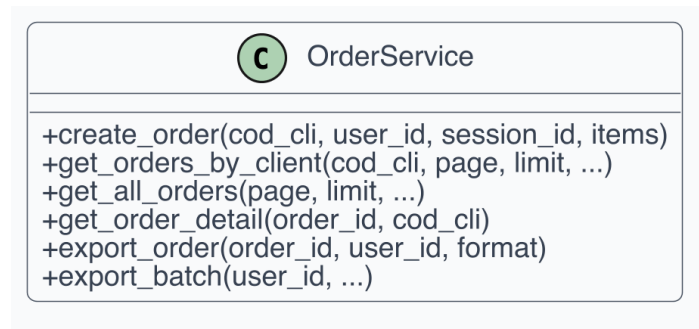


Figura 25: UML della classe `OrderService`.

7.1.2.5 TicketService **Descrizione della classe:** implementa l'intero ciclo di vita del ticket di assistenza, inclusa messaggistica, lock/unlock e notifiche SSE.

Descrizione degli attributi: nessun attributo esplicitato nel diagramma.

Descrizione dei metodi:

- `create_ticket(session_id: int, cod_cli: int)`: crea un nuovo ticket.
- `get_open_tickets()`: recupera i ticket aperti o in lavorazione.
- `get_ticket_by_id(ticket_id: int)`: recupera un ticket per ID.
- `get_ticket_by_session(session_id, cod_cli)`: recupera il ticket di una sessione.
- `lock_ticket(ticket_id, operator_id)`: assegna l'operatore (con notifica SSE).
- `unlock_ticket(ticket_id)`: rilascia il lock operatore.
- `close_ticket(ticket_id, closed_by)`: chiude definitivamente il ticket.
- `close_ticket_by_session(session_id, closed_by, cod_cli)`: chiude il ticket via sessione.
- `send_chat_message(ticket_id, content)`: invia messaggio operatore nel ticket (con SSE).
- `save_customer_message(session_id, content)`: salva messaggio cliente nel ticket.
- `get_platform_usage_overview(days)`: metriche aggregate per dashboard.

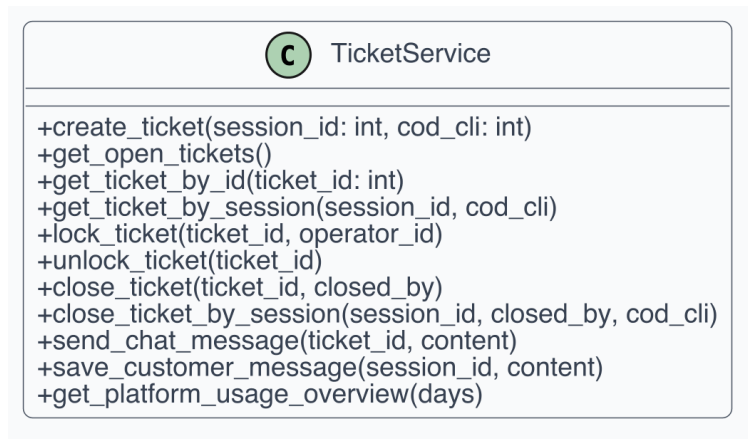


Figura 26: UML della classe **TicketService**.

7.1.2.6 FeedbackService **Descrizione della classe:** presidia la raccolta e la rimozione dei feedback qualitativi degli utenti sulle risposte AI.

Descrizione degli attributi: il diagramma non esplicita attributi interni.

Descrizione dei metodi:

- **save_feedback(message_id, user_id, is_positive, reason_category, comment):** salva il feedback nel database.
- **delete_feedback(message_id, user_id):** rimuove un feedback esistente.

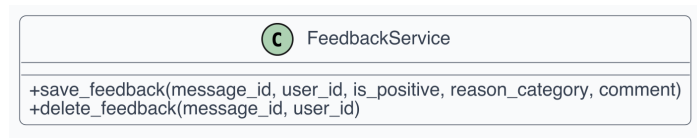


Figura 27: UML della classe **FeedbackService**.

7.1.2.7 ClientService **Descrizione della classe:** implementa la logica di business di ricerca clienti per ragione sociale o codice.

Descrizione degli attributi: il diagramma non esplicita attributi interni.

Descrizione dei metodi:

- **search_clients(query: str, limit: int):** cerca clienti nel database delegando al repository.

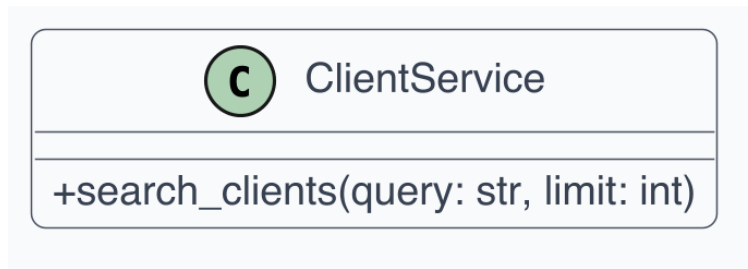


Figura 28: UML della classe `ClientService`.

7.1.2.8 ProductService **Descrizione della classe:** incapsula la logica per la ricerca prodotti nel catalogo, filtrata per assortimento cliente.

Descrizione degli attributi: il diagramma non esplicita attributi interni.

Descrizione dei metodi:

- `search_products(query: str, cod_cli: int, limit: int)`: cerca prodotti filtrati per assortimento cliente.



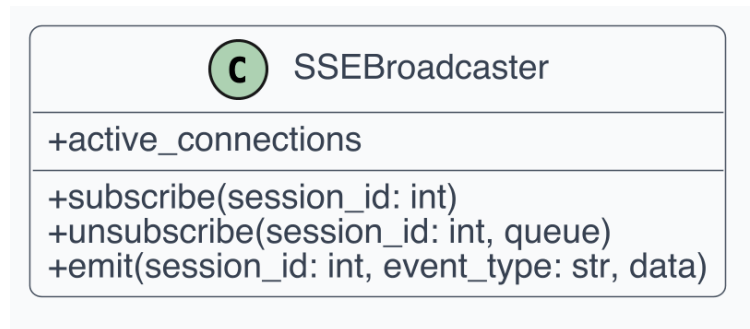
Figura 29: UML della classe `ProductService`.

7.1.2.9 SSEBroadcaster **Descrizione della classe:** singleton asyncio per il dispatch e la propagazione degli eventi real-time (SSE) a tutti i subscriber connessi, identificati per `session_id`.

Descrizione degli attributi: il diagramma non esplicita attributi persistenti per questa classe.

Descrizione dei metodi:

- `subscribe(session_id: int)`: registra un nuovo subscriber per la sessione.
- `unsubscribe(session_id: int, queue)`: rimuove uno specifico subscriber.
- `emit(session_id: int, event_type: str, data)`: broadcast di un evento a tutti i subscriber della sessione.
- `active_connections`: proprietà che conta le connessioni SSE attive totali.

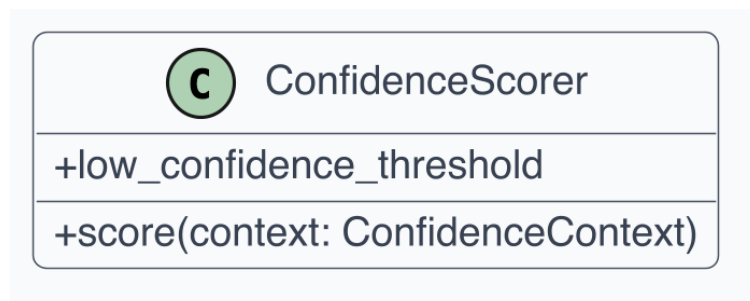
Figura 30: UML della classe **SSEBroadcaster**.

7.1.2.10 ConfidenceScorer **Descrizione della classe:** calcola il punteggio di affidabilità (Confidence Score) delle proposte AI, combinando segnali di storico, similarità, tipo di match, intento e quantità.

Descrizione degli attributi: il diagramma non esplicita attributi persistenti.

Descrizione dei metodi:

- `score(context: ConfidenceContext)`: calcola i punteggi di confidence per tutti i prodotti nel contesto.
- `low_confidence_threshold`: proprietà che restituisce la soglia sotto cui un articolo viene segnalato.

Figura 31: UML della classe **ConfidenceScorer**.

7.1.3 Core Domain (Entities & Value Objects)

7.1.3.1 Cliente **Descrizione della classe:** rappresenta il cliente commerciale autenticato e abilitato all'inserimento ordini.

Descrizione degli attributi:

- `id`: identificativo univoco (usato per join interne).
- `rag_soc`: denominazione legale dell'azienda cliente.
- `email`: contatto email principale per autenticazione/comunicazioni.

Descrizione dei metodi: non sono definiti metodi nel diagramma per questa entita.

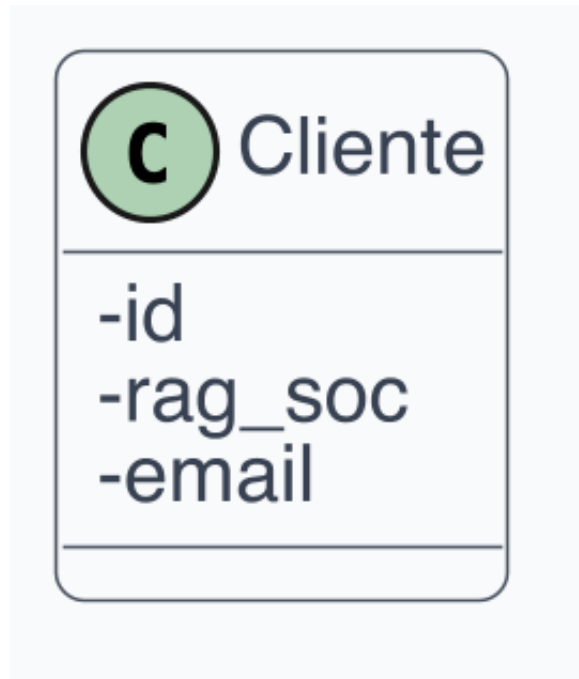


Figura 32: UML della classe `Cliente`.

7.1.3.2 Operatore **Descrizione della classe:** identifica l'utente interno che prende in carico ticket e supervisiona i casi critici.

Descrizione degli attributi:

- `id`: identificativo univoco operatore.
- `nome`: nominativo operatore.

Descrizione dei metodi: non sono definiti metodi nel diagramma.

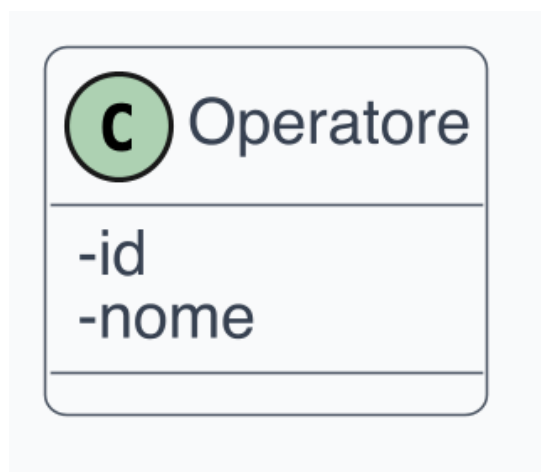


Figura 33: UML della classe `Operatore`.

7.1.3.3 Order **Descrizione della classe:** rappresenta l'ordine consolidato pronto per storicizzazione ed esportazione verso ERP.

Descrizione degli attributi:

- **id:** identificativo ordine.
- **cod_cli:** riferimento al cliente proprietario dell'ordine.
- **created_at:** timestamp di emissione.
- **righe:** collezione delle righe ordine derivate dal carrello.

Descrizione dei metodi: nel diagramma non sono mostrati metodi per questa entita.

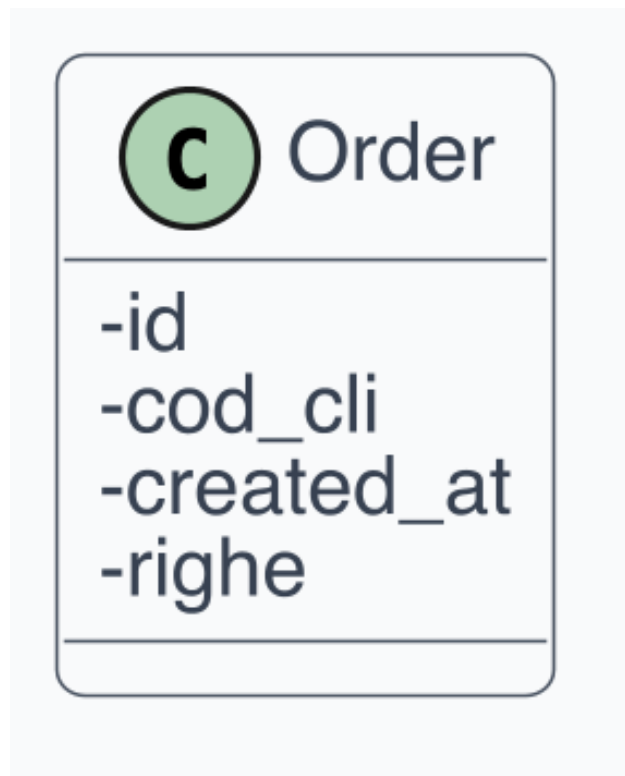


Figura 34: UML della classe **Order**.

7.1.3.4 Carrello **Descrizione della classe:** modella lo stato temporaneo dell'ordine in costruzione per una sessione chat.

Descrizione degli attributi:

- **session_id:** identificativo sessione conversazionale associata.
- **cod_cli:** cliente a cui il carrello appartiene.
- **righe:** elenco righe prodotto correnti.

Descrizione dei metodi:

- `calcolaTotalePezzi()`: restituisce la somma totale delle quantita nel carrello.

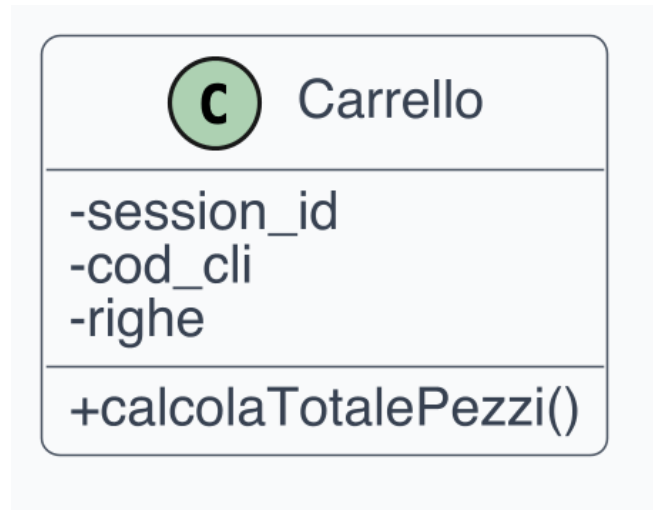


Figura 35: UML della classe **Carrello**.

7.1.3.5 RigaCarrello **Descrizione della classe:** rappresenta una singola posizione prodotto-quantita nel carrello.

Descrizione degli attributi:

- `cod_art`: riferimento all'articolo di catalogo.
- `quantity`: numero unita richieste.
- `confidence`: affidabilita della proposta AI su quella riga.

Descrizione dei metodi: non sono definiti metodi nel diagramma.

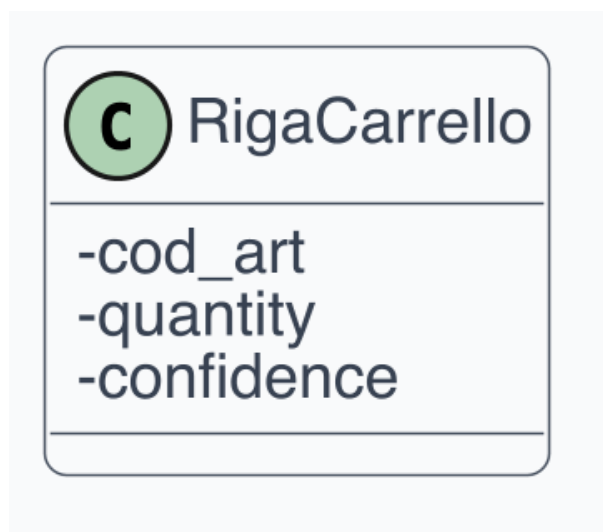


Figura 36: UML della classe **RigaCarrello**.

7.1.3.6 Session **Descrizione della classe:** mantiene il contesto di conversazione tra cliente e sistema durante la composizione ordine.

Descrizione degli attributi:

- **id:** identificativo sessione.
- **user_id:** identificativo dell'utente associato.
- **status:** stato della sessione.
- **messages:** storico messaggi scambiati.

Descrizione dei metodi: non sono presenti metodi nel diagramma.

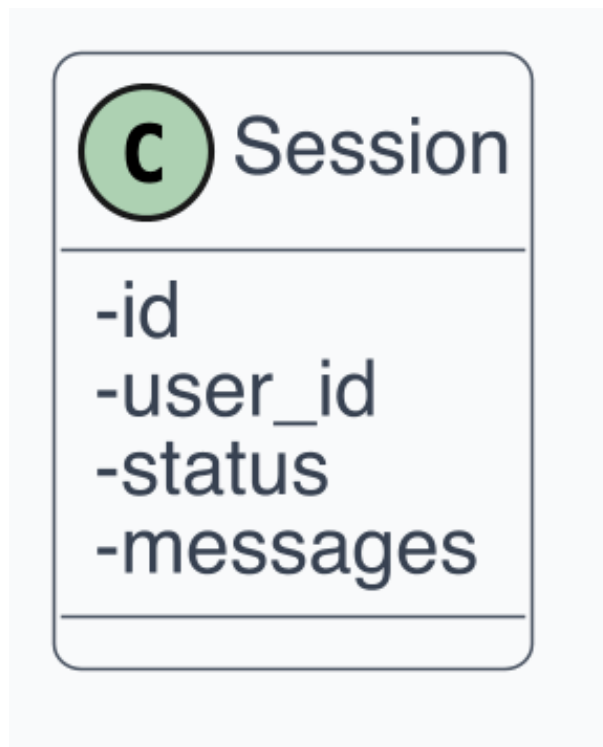


Figura 37: UML della classe **Session**.

7.1.3.7 Ticket **Descrizione della classe:** rappresenta una richiesta di supporto aperta per intervento umano.

Descrizione degli attributi:

- **id:** identificativo ticket.
- **cod_cli:** cliente che ha aperto il ticket.
- **stato:** stato corrente del ticket.
- **created_at:** data e ora di apertura.

Descrizione dei metodi: il diagramma non esplicita metodi per questa entita.

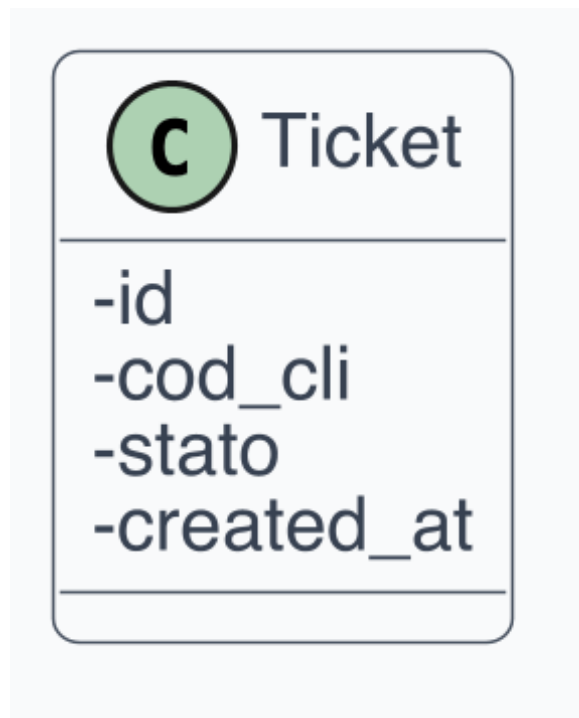


Figura 38: UML della classe Ticket.

7.1.3.8 LogAI Descrizione della classe: traccia input, output e correzioni legate alle elaborazioni AI per audit e miglioramento modello.

Descrizione degli attributi:

- **id:** identificativo del log.
- **inputOriginario:** testo/audio trascritto ricevuto dal cliente.
- **rispostaLLM:** proposta generata dal modello linguistico.
- **correzioneDelta:** differenza applicata manualmente da utente/operatore.
- **stato:** stato del log nel workflow di validazione.

Descrizione dei metodi: non sono definiti metodi nel diagramma.

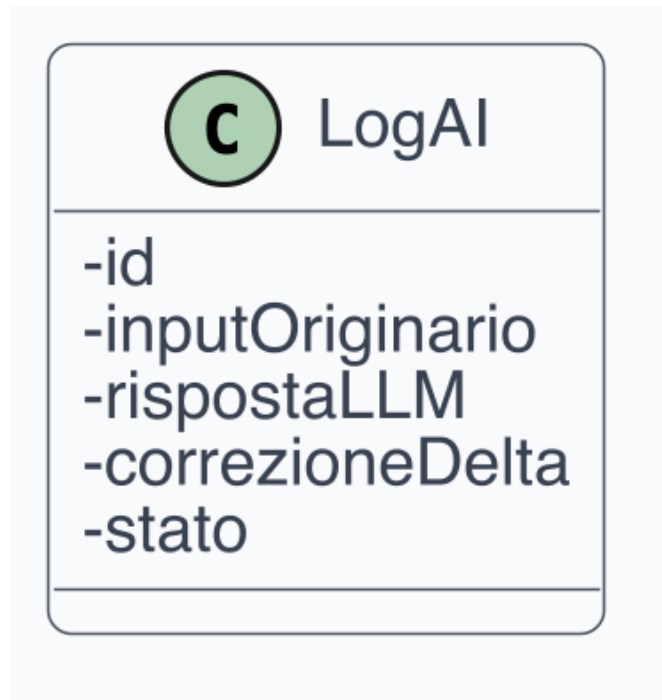


Figura 39: UML della classe LogAI.

7.1.4 Outbound Ports (Interfaces)

7.1.4.1 IUserRepository **Descrizione della classe/interfaccia:** definisce il contratto di accesso ai dati utente, credenziali e informazioni cliente.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `find_by_email(email: str)`: recupera l'utente associato all'email.
- `find_by_id(user_id: int)`: recupera l'utente per ID.
- `update_password(user_id, password_hash, password_salt)`: aggiorna hash e salt della password.
- `get_client_info(cod_cli: int)`: recupera ragione sociale e dati del cliente.
- `search_clients(query: str, limit: int)`: cerca clienti per ragione sociale o codice.
- `get_export_folder(user_id: int)`: recupera la cartella di esportazione.
- `set_export_folder(user_id: int, path)`: salva la cartella di esportazione.

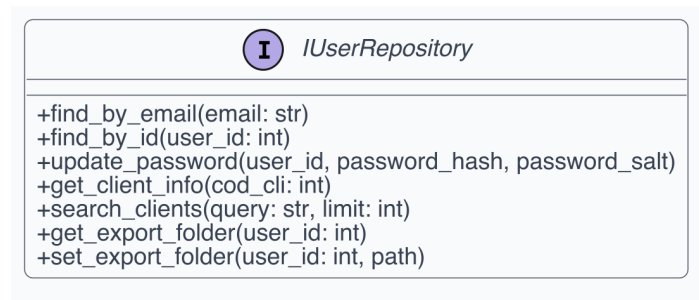


Figura 40: UML dell'interfaccia IUserRepository.

7.1.4.2 ISessionManager **Descrizione della classe/interfaccia:** astrae la persistenza di sessioni, messaggi e carrelli applicativi.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `get_active_session(user_id: int)`: recupera la sessione attiva dell'utente.
- `create_session(user_id: int)`: crea una nuova sessione.
- `get_messages(session_id: int)`: recupera i messaggi di una sessione.
- `save_message(session_id, sender, content, metadata)`: salva un messaggio.
- `get_cart_by_session(session_id: int)`: recupera il carrello della sessione.
- `get_cart(user_id: int)`: recupera il carrello dell'utente.
- `add_to_cart(user_id, cod_art, qta, source, ...)`: aggiunge un articolo al carrello.
- `add_to_cart_by_session(session_id, cod_art, qta, source, ...)`: aggiunge per sessione.
- `remove_from_cart(cart_item_id, user_id)`: rimuove un articolo.
- `remove_from_cart_by_session(cart_item_id, session_id)`: rimuove per sessione.
- `update_cart_quantity(cart_item_id, user_id, qta, source)`: aggiorna quantità.
- `update_cart_quantity_by_session(cart_item_id, session_id, qta, source)`: aggiorna per sessione.
- `clear_cart(user_id)`: svuota il carrello dell'utente.
- `clear_cart_by_session(session_id)`: svuota il carrello della sessione.

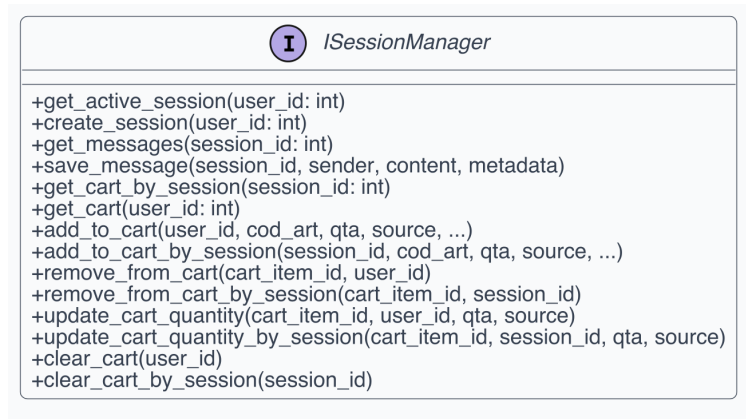


Figura 41: UML dell'interfaccia ISessionManager.

7.1.4.3 IOrderRepository **Descrizione della classe/interfaccia:** definisce le operazioni di persistenza e ricerca ordini.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `create_order(cod_cli, user_id, session_id, items)`: crea un ordine con le sue righe.
- `get_orders_by_client(cod_cli, page, limit, ...)`: recupera ordini del cliente con filtri.
- `get_all_orders(page, limit, ...)`: recupera tutti gli ordini (admin) con filtri.
- `get_order_detail(order_id, cod_cli)`: dettaglio completo di un ordine.
- `get_order_detail_any(order_id)`: dettaglio ordine senza filtro `cod_cli` (admin).

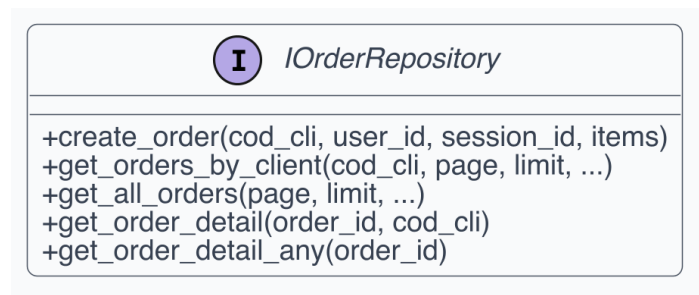


Figura 42: UML dell'interfaccia IOrderRepository.

7.1.4.4 ITicketRepository **Descrizione della classe/interfaccia:** astrae persistenza e query dei ticket di assistenza.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `create_ticket(session_id, cod_cli)`: crea un nuovo ticket.

- `get_ticket_by_session(session_id, cod_cli)`: recupera il ticket per sessione.
- `get_open_tickets()`: ritorna tutti i ticket aperti o in lavorazione.
- `get_ticket_by_id(ticket_id)`: recupera un ticket per ID.
- `lock_ticket(ticket_id, operator_id)`: imposta lo stato in `_lavorazione`.
- `unlock_ticket(ticket_id)`: rilascia il lock.
- `close_ticket(ticket_id)`: chiude il ticket definitivamente.
- `get_platform_usage_overview(days)`: metriche aggregate.
- `get_last_message_time(session_id)`: timestamp dell'ultimo messaggio.
- `send_message(session_id, sender, content)`: salva un messaggio nella chat.

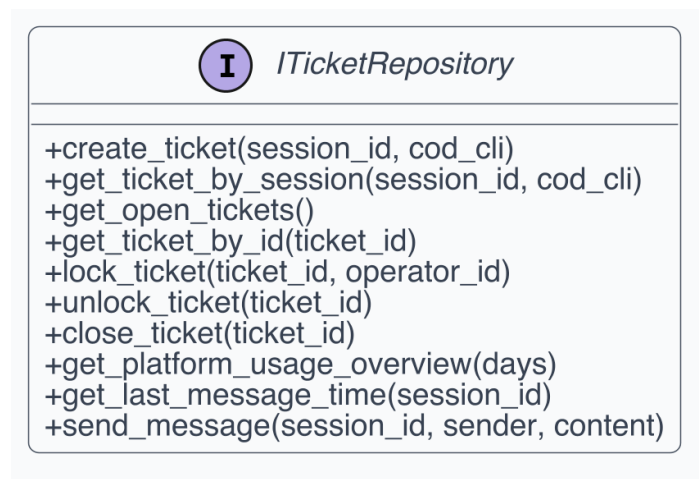


Figura 43: UML dell'interfaccia `ITicketRepository`.

7.1.4.5 IAIClient **Descrizione della classe/interfaccia:** formalizza il contratto verso i servizi di AI generativa, speech-to-text e analisi immagini.

Descrizione degli attributi: non applicabile (interfaccia senza stato).

Descrizione dei metodi:

- `transcribe_audio(audio_file: IO[bytes], filename: str)`: converte audio in testo.
- `interpret_intent(text, context, system_prompt, model)`: classifica intenti e produce decisione AI.
- `generate_embedding(text, model)`: calcola embedding vettoriale del testo.
- `analyze_image(image_base64, conversation_history, system_prompt)`: analizza un'immagine con GPT-5.4 Vision.

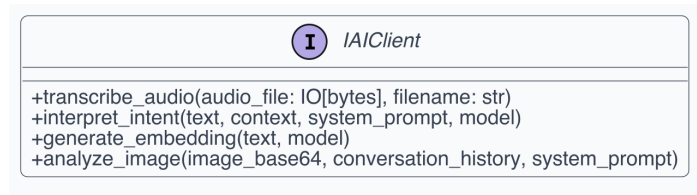


Figura 44: UML dell'interfaccia IAIClient.

7.1.5 Infrastructure Adapters

7.1.5.1 PostgresAdapter **Descrizione della classe:** implementa le porte IUserRepository, ISessionManager, IOrderRepository e ITicketRepository, interagendo direttamente con PostgreSQL tramite psycopg2 e connection pooling. Essendo un Adapter, svolge il ruolo cruciale di trasformare i tuple-record grezzi del database in veri e propri oggetti applicativi e DTO richiesti dalle interfacce del dominio.

Descrizione degli attributi:

- **pool:** istanza di SimpleConnectionPool per la gestione ottimizzata delle connessioni al database.

Descrizione dei metodi:

- **find_by_email(email):** implementa IUserRepository — recupera utente per email.
- **get_active_session(user_id):** implementa ISessionManager — recupera sessione attiva.
- **create_order(cod_cli, user_id, session_id, items):** implementa IOrderRepository — crea ordine.
- **create_ticket(session_id, cod_cli):** implementa ITicketRepository — crea ticket.
- **search_products(query, cod_cli, limit):** ricerca prodotti nell'assortimento cliente.
- **find_product_by_name(name, cod_cli):** ricerca prodotto per nome testuale.

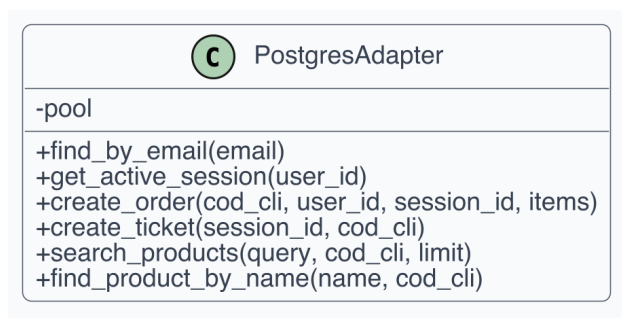


Figura 45: UML della classe PostgresAdapter.

7.1.5.2 OpenAIAdapter **Descrizione della classe:** realizza il contratto `IAIClient` mappando chiamate di dominio verso le API OpenAI (GPT-5.4, Whisper, Embeddings, Vision).

Descrizione degli attributi: il diagramma non esplicita attributi di configurazione (caricati da config).

Descrizione dei metodi:

- `transcribe_audio(audio_file, filename)`: trascrizione audio via Whisper con prompt specifico per il dominio Ergon.
- `interpret_intent(text, context, system_prompt, model)`: invoca il modello LLM con sistema prompt e contesto.
- `generate_embedding(text, model)`: genera un embedding vettoriale per il testo fornito.
- `analyze_image(image_base64, conversation_history, system_prompt)`: analizza immagini con GPT-5.4 Vision e restituisce OCR strutturato.

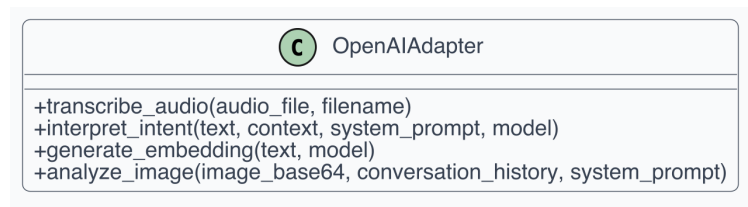


Figura 46: UML della classe `OpenAIAdapter`.

8 Architettura di Dettaglio - Frontend

Questa sezione descrive nel dettaglio l'architettura dei moduli del frontend. Per mantenere la coerenza formale con il backend e fornire un'analisi rigorosa seppur nel paradigma Component-Based, i costrutti fondamentali come Service, Context Provider e Hook Custom vengono esplorati analiticamente come singoli moduli ("classi").

8.1 Servizi di Accesso Dati (Service Facade)

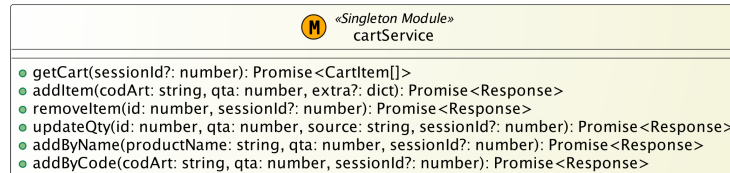
8.1.0.1 cartService **Descrizione della classe:** modulo Singleton che funge da Facade per la comunicazione asincrona col backend in materia di carrello. Abstrae i dettagli della libreria `ApiClient` (HTTP fetch e proxying).

Descrizione degli attributi: il modulo non possiede attributi di stato interni, in quanto il dominio del carrello è demandato al `CartContext`.

Descrizione dei metodi:

- `getCart(sessionId?)`: recupera il carrello attivo formattando il DTO in un array `CartItem[]`.
- `addItem(codArt, qta, extra?)`: aggiunge un nuovo articolo al carrello della sessione corrente.
- `removeItem(id, sessionId?)`: rimuove una riga ordine specifica.

- `updateQty(id, qta, source, sessionId?)`: aggiorna le quantità tenendo traccia dell'origine dell'evento (es. "customer" o "ai").
- `addByName(productName, qta, sessionId?)`: aggiunge elementi effettuando lookup testuale pilotato dal backend.
- `addByCode(codArt, qta, sessionId?)`: aggiunta precisa di elementi tramite `cod_art`.

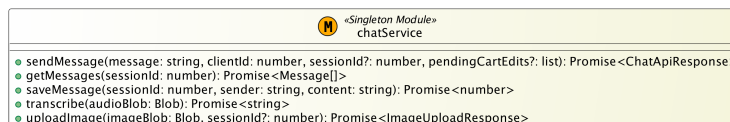
Figura 47: UML del modulo `cartService`.

8.1.0.2 chatService **Descrizione della classe:** modulo per incapsulare la complessa interazione utente-assistente e delegarla agli end-point API deputati all'AI.

Descrizione degli attributi: puramente stateless.

Descrizione dei metodi:

- `sendMessage(...)`: invia stringhe all'LLM e processa la risposta applicativa composita (include `ChatApiResponse` con intenzioni e modifiche carrello).
- `getMessages(sessionId)`: ripristina la persistenza dei messaggi in RAM recuperandola nel formato di dominio `Message[]`.
- `saveMessage(sessionId, sender, content)`: memorizza unilateralmente un messaggio offline.
- `transcribe(audioBlob)`: instrada il media blob verso Whisper (Backend API).
- `uploadImage(imageBlob, sessionId?)`: gestisce l'upload asincrono dell'immagini per analisi OCR.

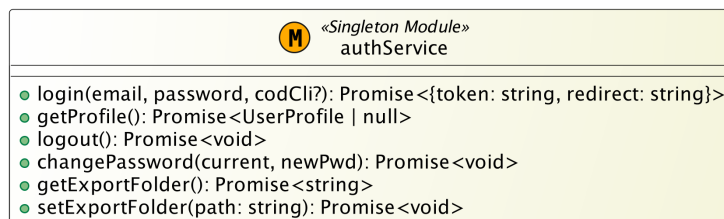
Figura 48: UML del modulo `chatService`.

8.1.0.3 authService **Descrizione della classe:** gestore dell'identità. Offre metodi per la pre-autenticazione, la negoziazione del token JWT con cookie HTTPOnly e il recupero del ruolo operatore o cliente.

Descrizione degli attributi: non porta con sé stati locali del token (questo per via dell'utilizzo di cookie gestiti dal browser o dal server Next.js).

Descrizione dei metodi:

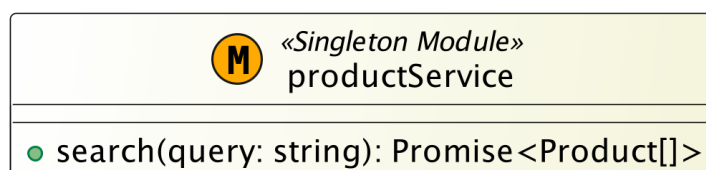
- `login(email, password, codCli?)`: negozia le credentials per ricevere session cookies e path di reindirizzamento.
- `getProfile()`: ritorna un'interfaccia `UserProfile` tipizzata.
- `logout()`: invoca la distruzione di sessione back-end e locale.
- `changePassword(current, newPwd)`: routine protetta di aggiornamento credenziali.
- `getExportFolder()`: recupera l'albero di directory d'esportazione dall'istanza utente.
- `setExportFolder(path)`: applica le preferenze relative all'export degli ordini da file system.

Figura 49: UML del modulo `authService`.

8.1.0.4 productService **Descrizione della classe:** modulo che gestisce l'interfacciamento con le logiche del backend per la navigazione del catalogo prodotti. **Descrizione degli attributi:**

stateless. **Descrizione dei metodi:**

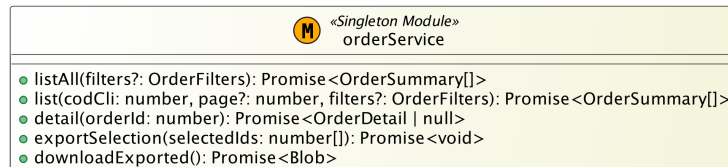
- `search(query)`: riceve input parziali, codifica l'URL e interroga il db vector/keywords restituendo una `Promise<Product[]>`.

Figura 50: UML del modulo `productService`.

8.1.0.5 orderService **Descrizione della classe:** esegue la facade per tutte le view relative allo storico ordini (cliente) o alla dashboard panoramica (backoffice operatore). **Descrizione degli attributi:** stateless. **Descrizione dei metodi:**

- `listAll(filters?)`: restituisce l'aggregato ordini globale.
- `list(codCli, page?, filters?)`: restituisce la pagina di ordini di uno specifico cliente.

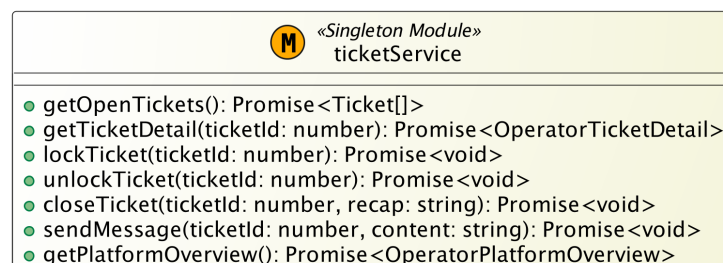
- `detail(orderId)`: recupera interamente un `OrderDetail` inclusivo di righe carrello.
- `exportSelection(selectedIds)`: inoltra una richiesta massiva al server per l'export su disco.
- `downloadExported()`: avvia il download del buffer binario (Zip/Excel) precedentemente elaborato.

Figura 51: UML del modulo `orderService`.

8.1.0.6 ticketService **Descrizione della classe:** facade utilizzata dagli operatori del Customer Care per la gestione delle casistiche delegate dal Bot. **Descrizione degli attributi:** stateless.

Descrizione dei metodi:

- `getOpenTickets()`: popola la tabella principale del backoffice con ticket inevasi.
- `getTicketDetail(ticketId)`: recupera i dati del singolo ticket e della chat utente annessa.
- `lockTicket(ticketId)` / `unlockTicket(ticketId)`: transazioni di locking competitivo per presa in carico esclusiva dell'operatore.
- `closeTicket(ticketId, recap)`: risolve il ticket inserendo un recap di chiusura.
- `sendMessage(ticketId, content)`: inietta un messaggio asincrono bypassando l'AI per comunicare con l'utente target.
- `getPlatformOverview()`: fornisce le metriche statistiche e l'array di andamento per la root della dashboard.

Figura 52: UML del modulo `ticketService`.

8.1.0.7 sessionService & Altri (feedbackService, healthService, apiClient) **Descrizione della classe:** raggruppamento di moduli minori per la diagnostica, metriche e la sessione esplicita. Da includere la libreria fondamentale apiClient. **Descrizione degli attributi:** le utilities

interne mantengono scope protetti sul modulo originario. **Descrizione dei metodi:**

- `sessionService.getActive()`: rintracciabilità dell’handshake primario.
- `sessionService.create()`: crea esplicitamente una sessione per uno specifico ID cliente in assenza di cookie.
- `feedbackService.submit(messageId, isPositive, reason?, comment?)`: inoltro del rating e feed di rinforzo sul modello.
- `healthService.check()`: pooling verso servizi esterni (status AI).
- `apiClient.apiFetch(path, init?)`: override globale per iniezione di cookie/CORS preflight.
- `apiClient.getApiErrorMessage(response, fallback)`: standardizza i payload JSON per l’estrazione granulare dell’errore (risoluzione 422 vs 500).
- `apiClient.buildApiUrl(path)`: prefissa consistentemente con lo snippet basePath.
- `apiClient.getApiBaseUrl()`: switcha gli use states ambientali (Dev locale proxata Next.js versus FastAPI puro).

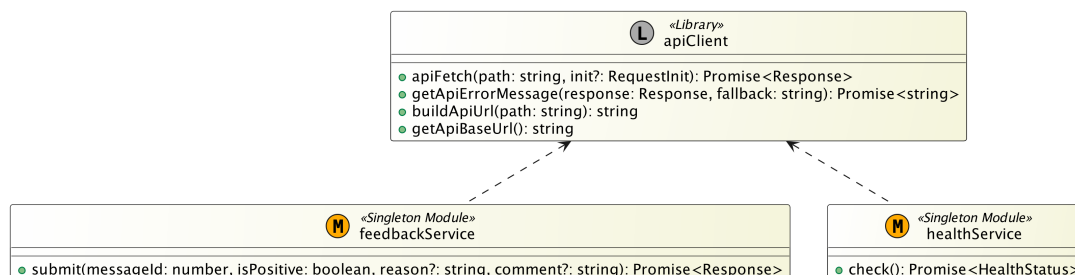


Figura 53: UML dei moduli minori e della libreria apiClient.

8.2 Contesti e Stato Applicativo (React Contexts)

8.2.0.1 SessionContext **Descrizione della classe:** Provider architetturale React che isola la cronologia logica di interazione (Bot e Umano) per l’utente, governando lo stato dinamico e disaccoppiandolo dai componenti visuali. Il two-way binding dello stato è risolto implicitamente e automaticamente dai re-render del Virtual DOM di React al mutare del Context (implementazione concreta dell’Observer pattern applicata all’UI).

Descrizione degli attributi:

- `sessionId`: identificativo univoco della transazione corrente associata a un cliente.
- `chatMessages`: array di storicizzazione in RAM di dominio `Message[]`.
- `showNewSessionModal`: flag di overlay per interrompere / generare sessioni.

Descrizione dei metodi:

- `initSession()`: richiede l'inizializzazione server di una track.
- `handleNewSession()` / `confirmNewSession()`: orchestra l>alert state.
- `cancelNewSession()`: annulla la creazione di una nuova sessione dismissing il modale protettivo.
- `setChatMessages(messages)`: dispatcher esposto per l'aggiornamento dei widget sottoscritti nella gerarchia React.

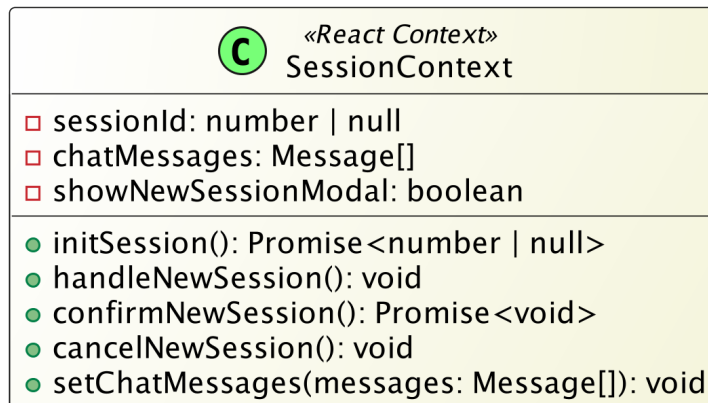


Figura 54: UML del Provider SessionContext.

8.2.0.2 CartContext **Descrizione della classe:** React Provider core del sistema, accentra e consolida le logiche di aggiornamento *Optimistic UI*.

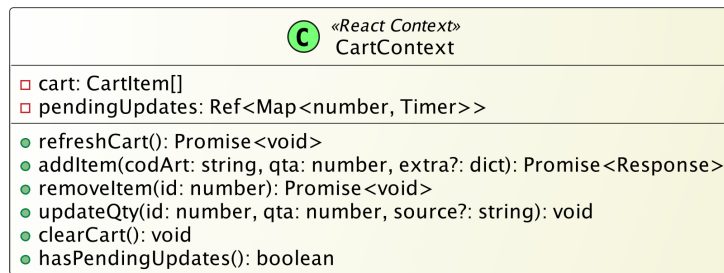
Descrizione degli attributi:

- `cart`: iterabile di articoli correnti (`CartItem[]`).
- `pendingUpdates`: Ref di memoria interno che mappa ID prodotto su specifici Timer di *Debounce map*, per rinviare sincronizzazioni a basso delta temporale dirette al backend.

Descrizione dei metodi:

- `refreshCart()`: innesca un overlay pulito scaricando la source of truth remota.
- `addItem(codArt, qta, extra?)`: gestore accodamento, triggera implicitamente il Refresh state.
- `removeItem(id)`: annulla l'oggetto da visualizzare.
- `updateQty(id, qta, source?)`: gestisce l'incremento di UI ottimistica, sopprimendo e ritardando l'effettiva request a 600ms per salvaguardare le performance.
- `clearCart()`: ripristina la referenza globale svuotando passivamente gli array di visualizzazione.

- `hasPendingUpdates()`: esegue una size introspection sul map del timer (es. per bloccare la redirect di UI quando ci sono flussi non confermati).

Figura 55: UML del Provider `CartContext`.

8.3 Flussi Asincroni (Custom Hooks)

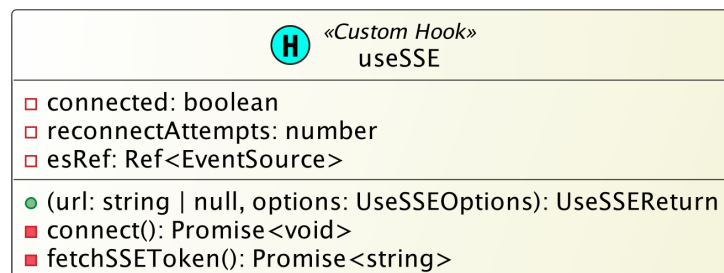
8.3.0.1 useSSE **Descrizione della classe:** Custom Hook isolato il cui scopo è gestire la negoziazione e la latenza dei `Server-Sent Events` con policy reattive verso l'interfaccia.

Descrizione degli attributi:

- `connected`: booleano derivato in tempo reale.
- `reconnectAttempts`: conteggio incrementale dei fallimenti via backoff esponenziale.
- `esRef`: `EventSource` raw instance allocata non-reattivamente.

Descrizione dei metodi:

- `connect()`: routine auto-rigenerativa preposta per il bootstrap o retrail di una stream socket.
- `fetchSSEToken()`: interfaccia il proxy bridge per ricavare credenziali sicure short-lived da appender in HTTP GET.
- `Hook call()`: accetta un URL e interfacce di callback (`UseSSEOptions`) restituendo lo stato esposto (`UseSSEReturn`). Destinato all'integrazione dentro i Provider sopracitati per emettere messaggi su socket asincrona diretta.

Figura 56: UML del modulo hook `useSSE`.

8.3.0.2 useHealthCheck / useProductSearch **Descrizione della classe:** moduli logici di corredo per separare lo stato del componente da computazioni esterne (la prima estendendo l'app sull'integrità del LLM, la seconda agendo da motore di query sul DB testuale/vettoriale riducendone il bounce).

Descrizione degli attributi (Stati reattivi interni):

- **status:** enum *loading* | *ok* | *error*.
- **searchTerm, results, isSearching:** payload di paginazione sui codici articolo.

Descrizione dei metodi:

- **check(hook call):** accetta cadenze (*intervalMs*).
- **doSearch(term):** incapsula un debounce locale al client prima di trasmettere tramite *productService*.

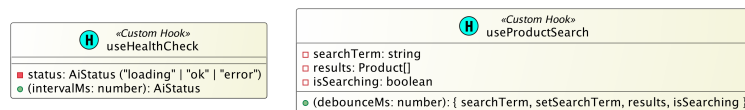
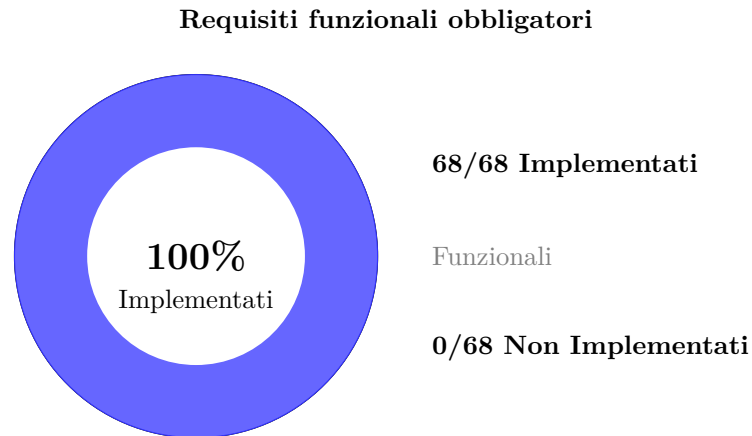


Figura 57: UML dei restanti custom hook del frontend.

9 Stato dei requisiti funzionali

Questa sezione riporta lo stato di implementazione dei requisiti funzionali derivati dall'ultima versione PB dell'Analisi dei Requisiti (v2.2). Per questa baseline, lo stato di ciascun requisito è indicato nella colonna **Stato** e tutti i requisiti obbligatori sono stati IMPLEMENTATI.



Nota: All'interno di questa baseline, tutti i 68 requisiti funzionali obbligatori sono stati completamente implementati e testati.

Figura 58: Riepilogo stato di implementazione dei requisiti funzionali.

ID	Importanza	Descrizione	Stato
RF-OB_01	Obbligatorio	Il sistema deve permettere all'Utente di inserire il proprio indirizzo email nel campo dedicato per effettuare il login.	IMPLEMENTATO
RF-OB_02	Obbligatorio	Il sistema deve permettere all'Utente di inserire la propria password in modo oscurato tramite il campo dedicato.	IMPLEMENTATO
RF-OB_03	Obbligatorio	Il sistema deve fornire un pulsante o comando per confermare l'accesso e inviare le credenziali per la validazione.	IMPLEMENTATO
RF-OB_04	Obbligatorio	Il sistema deve mostrare un errore se l'email inserita non ha un formato valido.	IMPLEMENTATO
RF-OB_05	Obbligatorio	Il sistema deve mostrare un errore se l'email inserita non è presente nel database locale.	IMPLEMENTATO
RF-OB_06	Obbligatorio	Il sistema deve mostrare un errore se la combinazione di email e password è sbagliata.	IMPLEMENTATO
RF-OB_07	Obbligatorio	Dopo un login valido, il sistema deve creare la sessione e far accedere l'utente alla propria area (Cliente o Operatore).	IMPLEMENTATO
RF-OB_08	Obbligatorio	Il sistema deve fornire un pulsante per effettuare il logout, invalidando e chiudendo la sessione corrente lato server.	IMPLEMENTATO

ID	Importanza	Descrizione	Stato
RF-OB_09	Obbligatorio	Il sistema deve mostrare i dati aziendali del Cliente (ID e Ragione Sociale) in sola lettura.	IMPLEMENTATO
RF-OB_10	Obbligatorio	Il sistema deve rendere accessibile la pagina Area Personale tramite un elemento di navigazione dedicato, consentendo all'Utente di consultare le informazioni del proprio profilo in base al ruolo assegnato.	IMPLEMENTATO
RF-OB_11	Obbligatorio	Il sistema deve mostrare all'Utente il proprio indirizzo email in sola lettura.	IMPLEMENTATO
RF-OB_12	Obbligatorio	Il sistema deve esporre un comando di contatto che, alla sua attivazione, invochi il protocollo URI <i>mailto:</i> per aprire il client di posta elettronica nativo dell'Utente con il campo destinatario pre-compilato con l'email di supporto aziendale.	IMPLEMENTATO
RF-OB_13	Obbligatorio	Il sistema deve mostrare in tempo reale tutti i messaggi inviati dall'Utente nella chat corrente.	IMPLEMENTATO
RF-OB_14	Obbligatorio	Il sistema deve mostrare in tempo reale le risposte testuali scritte dall'AI o dall'Operatore di supporto.	IMPLEMENTATO
RF-OB_15	Obbligatorio	Il sistema deve fornire un campo di testo per l'inserimento e l'invio di messaggi testuali nella chat.	IMPLEMENTATO
RF-OB_16	Obbligatorio	Il sistema deve bloccare l'invio e mostrare un errore se il messaggio di testo supera il limite di caratteri.	IMPLEMENTATO
RF-OB_17	Obbligatorio	Il sistema deve permettere all'Utente la registrazione e l'invio di messaggi vocali tramite il microfono del dispositivo.	IMPLEMENTATO
RF-OB_18	Obbligatorio	Il sistema deve convertire l'audio in testo usando un modulo Speech-to-Text.	IMPLEMENTATO
RF-OB_19	Obbligatorio	Il sistema deve bloccare l'invio e mostrare un errore se il file audio pesa più di 10MB.	IMPLEMENTATO
RF-OB_20	Obbligatorio	Il sistema deve richiedere esplicitamente i permessi hardware necessari (microfono, fotocamera) al momento del primo utilizzo della relativa funzionalità e, in caso di rifiuto da parte dell'Utente, mostrare un messaggio informativo che spieghi l'impossibilità di procedere senza tali permessi.	IMPLEMENTATO
RF-OB_21	Obbligatorio	Il Cliente deve poter resettare la chat con l'AI e svuotare il carrello attuale.	IMPLEMENTATO

ID	Importanza	Descrizione	Stato
RF-OB_22	Obbligatorio	Il sistema deve disabilitare il comando di reset della sessione qualora sia presente un ticket di assistenza in stato “Aperto” o “In lavorazione” per la sessione corrente, impedendo all’Utente di azzerare il contesto durante un’assistenza attiva.	IMPLEMENTATO
RF-OB_23	Obbligatorio	L’AI (LLM) deve capire quali prodotti vuole l’utente e in che quantità leggendo/ascoltando il messaggio.	IMPLEMENTATO
RF-OB_24	Obbligatorio	Il sistema deve cercare i prodotti nel database usando la ricerca vettoriale (Vector Search).	IMPLEMENTATO
RF-OB_25	Obbligatorio	Il sistema deve aggiungere in automatico i prodotti trovati al carrello se il Confidence Score calcolato dall’AI supera la soglia definita in RP-OB_03.	IMPLEMENTATO
RF-OB_26	Obbligatorio	Vincolo Assortimento: Il sistema deve ignorare i prodotti che non fanno parte del catalogo assegnato a quel Cliente (tabella asscli).	IMPLEMENTATO
RF-OB_27	Obbligatorio	Vincolo Stato: Il sistema deve impedire l’aggiunta di prodotti segnati come “non disponibili” o “sospesi” nel database (anaart).	IMPLEMENTATO
RF-OB_28	Obbligatorio	Deduzione Formati: L’AI deve convertire automaticamente le quantità richieste nelle giuste unità di vendita (es. da bottiglie a casse).	IMPLEMENTATO
RF-OB_29	Obbligatorio	Il sistema deve fornire una barra di ricerca per cercare manualmente i prodotti nel catalogo tramite input testuale.	IMPLEMENTATO
RF-OB_30	Obbligatorio	Il sistema deve mostrare i risultati della ricerca in tempo reale (Live Search).	IMPLEMENTATO
RF-OB_31	Obbligatorio	Il sistema deve mostrare per ogni prodotto trovato: Nome, ID, Linea, Famiglia e Modalità di Vendita.	IMPLEMENTATO
RF-OB_32	Obbligatorio	L’Utente deve poter cliccare su un risultato per decidere quanti pezzi acquistarne.	IMPLEMENTATO
RF-OB_33	Obbligatorio	L’Utente deve poter modificare la quantità usando i tasti +/- o scrivendo direttamente il numero.	IMPLEMENTATO
RF-OB_34	Obbligatorio	Il sistema deve correggere in automatico la quantità se l’utente inserisce numeri negativi, zero o troppo grandi.	IMPLEMENTATO

ID	Importanza	Descrizione	Stato
RF-OB_35	Obbligatorio	L'Utente deve poter confermare l'aggiunta del prodotto al carrello della sessione corrente.	IMPLEMENTATO
RF-OB_36	Obbligatorio	Il sistema deve mostrare un avviso di "Carrello Vuoto" se non ci sono prodotti inseriti.	IMPLEMENTATO
RF-OB_37	Obbligatorio	Il sistema deve mostrare i dettagli logistici e le quantità di tutti i prodotti presenti nel carrello.	IMPLEMENTATO
RF-OB_38	Obbligatorio	L'Utente deve poter sbloccare la riga di un prodotto nel carrello per poterne modificare la quantità.	IMPLEMENTATO
RF-OB_39	Obbligatorio	L'Utente deve poter salvare la nuova quantità modificata per un prodotto nel carrello.	IMPLEMENTATO
RF-OB_40	Obbligatorio	L'Utente deve poter rimuovere definitivamente un articolo dal carrello.	IMPLEMENTATO
RF-OB_41	Obbligatorio	Il sistema deve disabilitare il pulsante per inviare l'ordine se il carrello è vuoto.	IMPLEMENTATO
RF-OB_42	Obbligatorio	L'Utente (Cliente o Operatore in subentro) deve poter chiudere il carrello e inviare l'ordine definitivo.	IMPLEMENTATO
RF-OB_43	Obbligatorio	Il sistema deve bloccare l'invio dell'ordine e segnalare gli errori se ci sono problemi con le regole del gestionale (es. prodotto fuori catalogo).	IMPLEMENTATO
RF-OB_44	Obbligatorio	Dopo l'invio dell'ordine, il sistema deve svuotare la sessione e reindirizzare l'Utente alla vista appropriata al proprio ruolo: il Cliente allo Storico Ordini, l'Operatore alla chiusura del ticket e dismissione della console di subentro.	IMPLEMENTATO
RF-OB_45	Obbligatorio	Il sistema deve creare un file JSON formattato contenente tutti i dati dell'ordine appena chiuso.	IMPLEMENTATO
RF-OB_46	Obbligatorio	Il Cliente deve poter visualizzare la lista dei propri ordini passati, ordinati per data.	IMPLEMENTATO
RF-OB_47	Obbligatorio	Il sistema deve mostrare per ogni ordine storico: ID, Data, totale degli articoli e una piccola anteprima dei prodotti.	IMPLEMENTATO
RF-OB_48	Obbligatorio	Il Cliente deve poter cliccare su un ordine passato per vederne tutti i dettagli logistici.	IMPLEMENTATO
RF-OB_49	Obbligatorio	Il sistema deve mostrare in sola lettura la chat originale tra utente e AI che ha generato quell'ordine specifico.	IMPLEMENTATO

ID	Importanza	Descrizione	Stato
RF-OB_50	Obbligatorio	Il Cliente deve poter richiedere l'aiuto di un operatore umano, mettendo in pausa l'AI.	IMPLEMENTATO
RF-OB_51	Obbligatorio	L'Operatore deve poter accedere alla dashboard con la lista dei Ticket Aperti (richieste d'aiuto).	IMPLEMENTATO
RF-OB_52	Obbligatorio	Nella lista dei ticket, il sistema deve mostrare da quanto tempo il Cliente sta aspettando una risposta.	IMPLEMENTATO
RF-OB_53	Obbligatorio	L'Operatore deve poter prendere in carico un ticket bloccandolo per gli altri colleghi (Lock) per aprire la console di assistenza.	IMPLEMENTATO
RF-OB_54	Obbligatorio	L'Operatore deve poter scrivere e inviare messaggi in chat direttamente al Cliente per fornirgli assistenza.	IMPLEMENTATO
RF-OB_55	Obbligatorio	L'Operatore deve poter usare la ricerca per aggiungere forzatamente prodotti al carrello del Cliente che sta aiutando.	IMPLEMENTATO
RF-OB_56	Obbligatorio	L'Operatore deve poter modificare le quantità o rimuovere prodotti dal carrello del Cliente.	IMPLEMENTATO
RF-OB_57	Obbligatorio	L'Operatore deve poter chiudere il ticket per segnare il problema come risolto e sbloccare la sessione (rimuovere il Lock).	IMPLEMENTATO
RF-OB_58	Obbligatorio	L'Utente (Cliente o Operatore) deve poter chiudere il ticket da solo se risolve il problema o non ha più bisogno di aiuto.	IMPLEMENTATO
RF-OB_59	Obbligatorio	L'Operatore deve poter accedere alla dashboard che mostra tutti gli ordini inviati da tutti i Clienti.	IMPLEMENTATO
RF-OB_60	Obbligatorio	L'Operatore deve poter cliccare su un ordine di qualsiasi cliente per aprirlo e vederne i dettagli.	IMPLEMENTATO
RF-OB_61	Obbligatorio	L'Operatore deve poter cercare del testo solo all'interno di una colonna specifica (es. cercare solo l'ID Ordine).	IMPLEMENTATO
RF-OB_62	Obbligatorio	L'Operatore deve poter ordinare le tabelle in modo crescente o decrescente cliccando sui nomi delle colonne.	IMPLEMENTATO
RF-OB_63	Obbligatorio	L'Operatore deve poter filtrare i dati delle tabelle scegliendo una data di inizio e di fine.	IMPLEMENTATO
RF-OB_64	Obbligatorio	Il sistema deve salvare nel database tutto lo storico delle chat e delle decisioni prese dall'AI.	IMPLEMENTATO

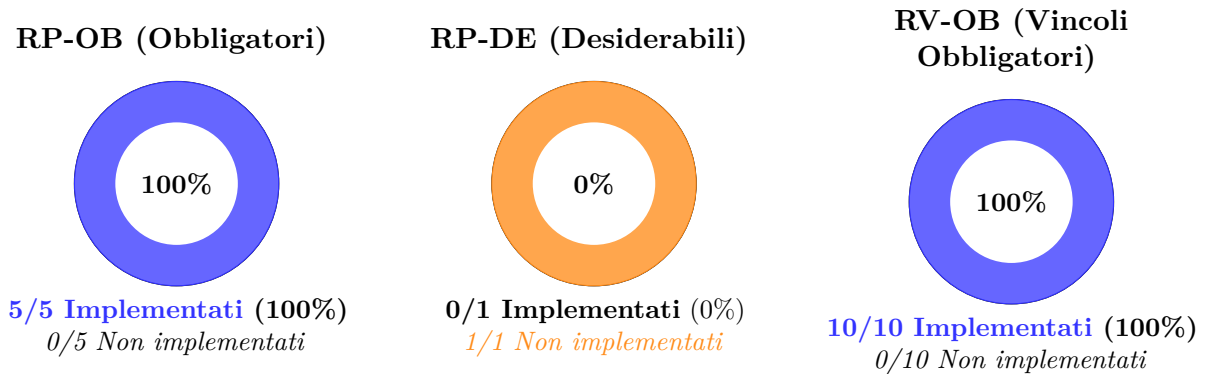
ID	Importanza	Descrizione	Stato
RF-OB_65	Obbligatorio	Il sistema deve mostrare la percentuale di sicurezza dell'AI (Confidence Score) solo nelle schermate dedicate all'Operatore.	IMPLEMENTATO
RF-OB_66	Obbligatorio	Il sistema deve mostrare schermate di caricamento (spinner o barre di avanzamento) durante le operazioni che richiedono un tempo di elaborazione superiore a 1 secondo.	IMPLEMENTATO
RF-OB_67	Obbligatorio	Le API del server (Backend) devono rifiutare qualsiasi richiesta priva di un token di accesso valido per la sessione corrente.	IMPLEMENTATO
RF-OB_68	Obbligatorio	Il sistema deve caricare i dati delle tabelle in modo incrementale tramite paginazione con pagine da massimo 50 record, per evitare il blocco dell'interfaccia in presenza di grandi volumi di dati.	IMPLEMENTATO
RF-DE_01	Desiderabile	Il sistema deve fornire un link per accedere alla pagina di recupero della password dimenticata.	NON IMPLEMENTATO
RF-DE_02	Desiderabile	L'Utente deve poter richiedere il reset della password inserendo la propria email.	NON IMPLEMENTATO
RF-DE_03	Desiderabile	Il sistema deve permettere all'Utente loggato di avviare la procedura per cambiare la propria password.	IMPLEMENTATO
RF-DE_04	Desiderabile	L'Utente deve poter inviare la vecchia password e la nuova password scelta per salvarla nel sistema.	IMPLEMENTATO
RF-DE_05	Desiderabile	Il sistema deve bloccare il cambio password se la vecchia password è errata o se le due nuove password inserite non coincidono.	IMPLEMENTATO
RF-DE_06	Desiderabile	Il sistema deve permettere l'invio di file immagine nella chat, mediante scatto fotografico o selezione dalla galleria del dispositivo.	IMPLEMENTATO
RF-DE_07	Desiderabile	Il sistema deve bloccare l'invio di immagini e mostrare un errore se il file pesa più di 5MB.	IMPLEMENTATO
RF-DE_08	Desiderabile	Il Cliente deve poter valutare la qualità delle risposte dell'AI lasciando un feedback (es. pollice su / pollice giù).	IMPLEMENTATO
RF-DE_09	Desiderabile	Se il Cliente dà un voto negativo all'AI, il sistema deve richiedere che indichi il tipo di errore (tramite tag predisposti).	IMPLEMENTATO

ID	Importanza	Descrizione	Stato
RF-DE_10	Desiderabile	Il Cliente deve poter aggiungere un testo libero per spiegare meglio il motivo dell'errore segnalato.	IMPLEMENTATO
RF-DE_11	Desiderabile	Il Cliente deve poter scaricare l'elenco dei propri ordini storici in un file formato CSV.	NON IMPLEMENTATO
RF-DE_12	Desiderabile	L'Operatore deve poter accedere alla dashboard per il controllo degli errori e l'addestramento dell'AI (Ottimizzazione AI).	IMPLEMENTATO
RF-DE_13	Desiderabile	Il sistema deve mostrare all'Operatore la lista degli errori segnalati dai clienti e le correzioni manuali fatte ai carrelli.	NON IMPLEMENTATO
RF-DE_14	Desiderabile	L'Operatore deve poter aprire un singolo errore AI per rileggere tutta la chat originale tra il Cliente e il bot.	NON IMPLEMENTATO
RF-DE_15	Desiderabile	L'Operatore deve poter approvare una segnalazione di errore, confermando che i dati verranno usati per riaddestrare l'AI.	NON IMPLEMENTATO
RF-DE_16	Desiderabile	L'Operatore deve poter scartare una segnalazione se non si tratta di un vero errore dell'AI (es. l'utente ha scritto male).	NON IMPLEMENTATO
RF-DE_17	Desiderabile	L'Operatore deve poter annullare l'approvazione di una segnalazione se l'aveva confermata per sbaglio (Rollback).	NON IMPLEMENTATO
RF-OP_01	Facoltativo	Il sistema deve permettere l'invio di messaggi d'ordine e la loro elaborazione tramite l'app di WhatsApp.	NON IMPLEMENTATO
RF-OP_02	Facoltativo	Il sistema deve permettere l'invio di messaggi d'ordine e la loro elaborazione tramite l'app di Telegram.	NON IMPLEMENTATO
RF-OP_03	Facoltativo	Il sistema deve generare grafici e statistiche generali sull'uso della piattaforma, mostrandoli all'Operatore.	IMPLEMENTATO

Tabella 14: Stato dei requisiti funzionali (fonte: Analisi dei Requisiti PB v2.2)

9.1 Riepilogo stato requisiti prestazionali e di vincolo

I requisiti prestazionali e di vincolo comprendono specifiche non-funzionali critiche per la qualità e l'usabilità del sistema. La seguente analisi distingue tra requisiti obbligatori (OB) e desiderabili (DE).



Categoria	Totale	Implementati	Non implementati
RP-OB	5	5	0
RP-DE	1	0	1
RV-OB	10	10	0

Tabella 15: Riepilogo implementazione requisiti prestazionali e di vincolo